CITS5504 Data Warehouse

Project 1 - Data Warehouse Design

2026 - 04 - 16

Authors:	
Jason Wong	24839132
Chalaka Nawarathna	24609438

1. Introduction

This project focuses on designing and implementing a data warehouse used for analysis and answering business queries, particularly with a focus on Australia. The main goal is to transform raw operational datasets into structured, multidimensional models that allow fast and efficient querying, analysis and decision-making. The project aims to create a schema that is intuitive, scalable and aligned with OLAP workflow, by applying dimensions modelling principles on airline datasets. Despite the availability of large airline datasets, some of it is often fragmented and inconsistent, making it difficult to perform analysis and extract meaningful insights. There needs to integrate these datasets into a structured data warehouse that can support fast querying, multidimensional analysis and association mining. This project should address this issue of transforming raw data into an optimized schema that enables effective analysis of passenger flights and demographics.

The given datasets consist of multiple source files containing information about passenger flights, airports, countries and regions. The data varies in granularity and structure, requiring cleaning, integration and transformation before being implemented in dimensional modelling. It is also essential to have a good understanding about the relationships and hierarchies in the dataset for defining the fact table, dimension tables and the concept hierarchies.

The data warehouse architecture follows a standard ETL process. The data will be first extracted and viewed to get a good understanding about the datasets. After, the data will be cleaned and preprocessed before joining all of the datasets into one. Lastly, we will then start loading in the dimensions and fact tables using the fully integrated dataset and create a star-schema. The final warehouse will consist of one fact table supported by several dimension tables, enabling OLAP operation such as roll-up, drill-down and slicing and dicing. This architecture will ensure efficient query performance and supports business query analysis.

Furthermore, we apply association mining to the integrated dataset. While OLAP can reveal trends through aggregation and dimensional exploration, association mining provides a complementary perspective by uncovering hidden patterns that may not be immediately visible through standard analysis. Specifically, the mining process was configured on Flight_Status as the RHS, giving us insights on the top-k rules that describe how airline demographics affect flight status outcome.

Overall, the data warehouse, combined with the OLAP and association mining application on the integrated dataset, provides a structured foundation for analysing flight patterns and demographic behaviors, which can be used to support operational and policy decisions. Airlines can use these insights to monitor performance and identify areas for improvement, and the Australian governments can leverage this to assess infrastructure needs and strengthen aviation oversight.

2. Data Warehouse Design

2.1. Identify Process being Modelled

The business process being modelled in this data warehousing project is the occurrences of passengers departing from an airport, where each instance represents a passenger departing from a specific airport on a specific date during 2022. The process identifies the fundamental interactions between passengers, airports, time and location attributes, providing the basis for analyzing travel behavior and airport utilization.

The main goal is to record these events rather than quantify them using operational metrics. Thus, the warehouse will employ a factless fact table to represent the relationship among the dimensions.

Moreover, The reason for this type of fact table is because the dataset captures occurrences of flight departures rather than numerical measures. The event has no natural numerical measures, so the table captures the relationships among the different dimensions. This design accurately represents the business process while still allowing for flexible analysis to count events and explore patterns across different attributes.

2.2. Determine the Grain

The grain of the fact table is defined by a single row, which represents the lowest level of detail for a business event. Based on the dataset, the grain is defined by one passenger departing from a specific airport origin, on a specific day and its flight status.

The reason for the chosen grain is because the dataset captures a passenger's flight information, the date they departed, the airport and location they departed from and the status of the flight. Since, the column labelled 'Arrival Airport' is an ETL error, the dataset cannot support origin-destination flight routes. Thus, the most accurate and meaningful event we can model is the occurrence of passengers departing from an origin airport on a specific date in 2022.

Lastly, defining this level of grain enables the warehouse to help support analysis across passenger demographics, airport utilization, date patterns and flight status variations. It also provides a foundation for flexible roll-ups along key concept hierarchies, ensuring that the fact table remains robust, expandable and capable of answering a wide range of business queries.

2.3. Dimensions Table Design

After defining the grain and business process, we will now identify the dimensions that will be used for the data warehouse. The dimensions are used to provide descriptive contexts for each passenger's departing event. The purpose of this is to support slicing, filtering and aggregation across demographics, time and location. Thus, providing a wide range of analytical queries for the fact table.

Dim_Passenger: Passenger dimension – contains demographic information about the departing passenger.

Column Name	Description
Passenger_Key(PK)	Surrogate key made from combining First Name, Last Name and Age of the passenger, thus enabling for unique passengers.
First_Name	Individual passenger's first name
Last_Name	Individual passenger's last name
Gender	Individual passenger's sex(Male or Female)
Age	Individual passenger's age
Age_Type	Age group of the passenger (e.g., Child, Young adult, Adult or Senior)

Dim_Nationality: Nationality dimension – contains the nationality of all unique passengers.

Column Name	Description
Nationality_ID(PK)	Unique identifier for each nationality
Nationality	The nationality of the passenger (e.g, Australian, Chinese, etc.)

Dim_Date: Date dimension – contains date attributes.

Column Name	Description
Date_ID(PK)	Unique identifier for each date
Departure_Date	The date each passenger departed for their flight
Year	The Year each passenger departed for their flight
Quarter	The Quarter each passenger departed for their flight
Month	The Month each passenger departed for their flight

Dim_Location: Location dimension – contains the geographic information of the airports.

Column Name	Description
Location_ID	Unique identifier for each location
Continent	The continent an airport is located
Country	The country an airport is stationed
Region	The specific region of the country

Dim_Airport: Airport dimension – contains the airport information, such as name and size.

Column Name	Description
Airport_ID	Unique identifier for each airport
Airport_Name	Name of the airport
Airport_Size	The size of the airport

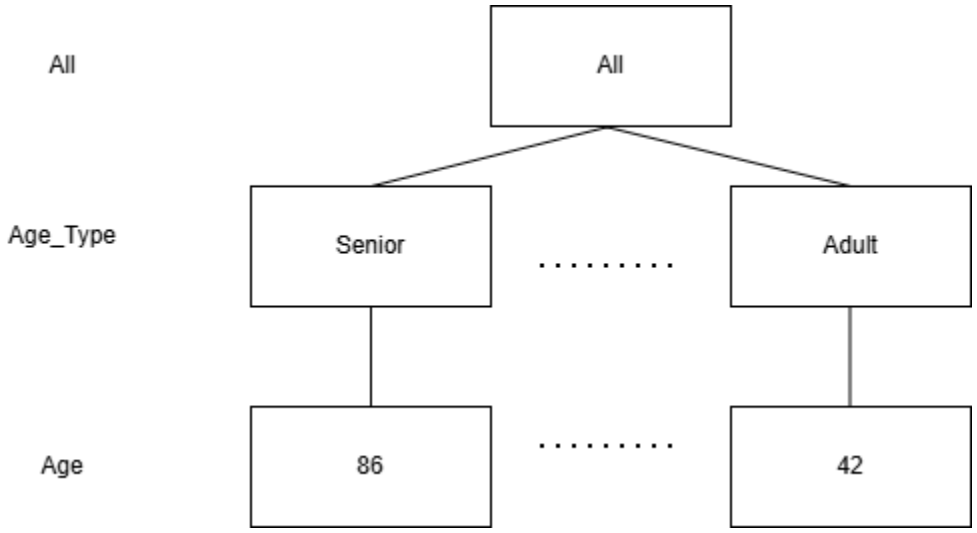
Dim _Status: contains the flight status for a specific flight a passenger is taking.

Column Name	Description
Status_ID	Unique identifier for each flight status
Flight_Status	The flight status (On-Time, Cancelled or Delayed)

Conceptual Hierarchies:

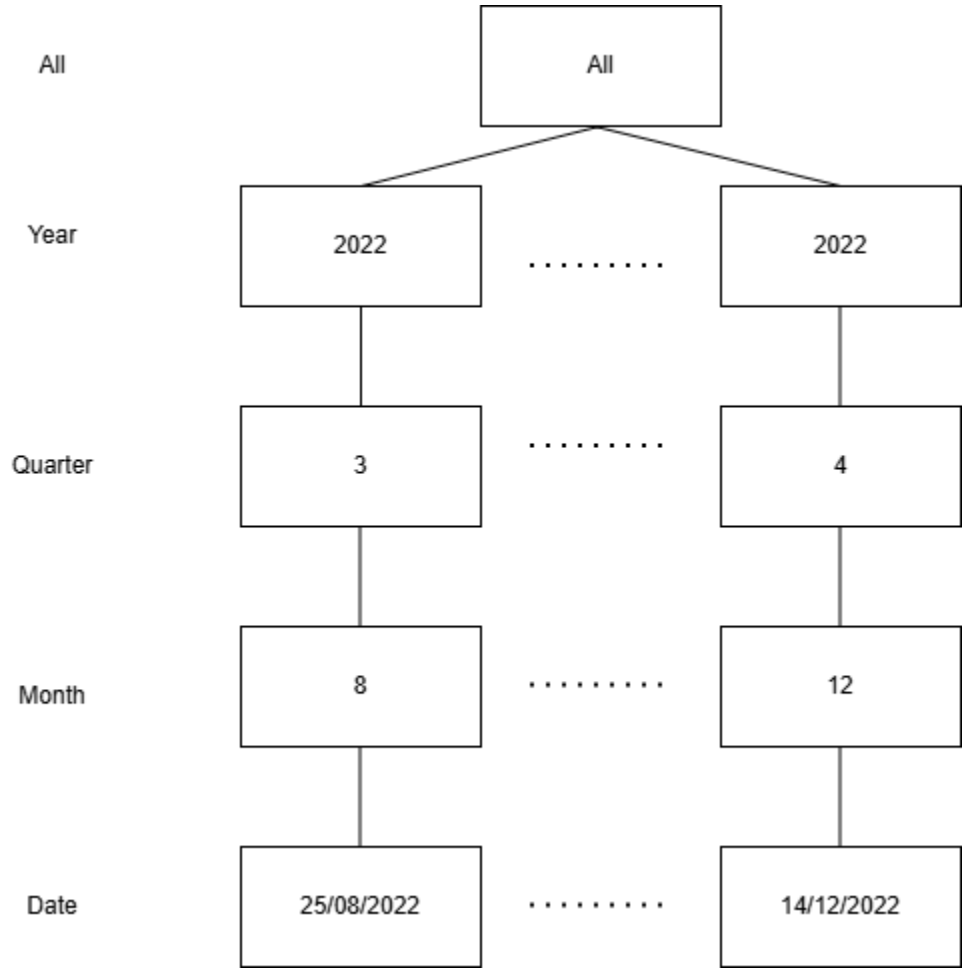
The dimensions in a data warehouse, the concept hierarchies are the levels of granularity within each dimension. This enables users to drill-down to more detailed data or to roll-up to a more generalized high-level view. These hierarchies are important as they support Online Analysis Process operations like slicing, dicing and aggregation. From the dimensions structured for this project, several hierarchies can be shown to allow users to apply these OLAP operations. We highlight the hierarchies below.

Age Concept Hierarchy:



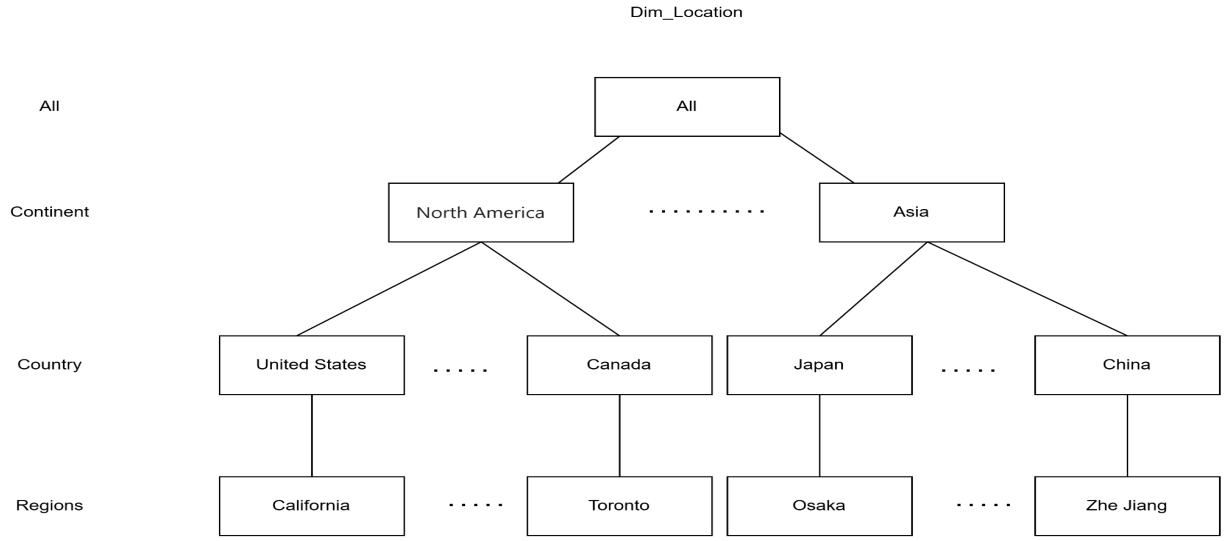
For the passenger dimension, there is a concept hierarchy for the age. The lowest level is the exact age for each passenger, we can roll it up to a broader stage, which is the age type, either they are a senior, young adult, adult or a child. This hierarchy allows us to look into different levels of detail, from an individual passenger's age to a more general category for each passenger's age.

Date Concept Hierarchy:



There is a concept hierarchy for the date. The lowest level is the departure date for each passenger, we can roll it up to a few broader stages, which are the month, quarter and year. This hierarchy allows us to look into different date details, from the month they departed, to the exact quarter. This can help analyse data that may not be noticeable on an exact date level, but maybe more visible in a grouped date level.

Dim_Location Concept Hierarchy:



There is a concept hierarchy for the location dimension. The lowest level is the regions for each country, we can roll it up to a few broader stages, which are the country each region belongs to, and the continent the country belongs to. This hierarchy allows us to look into different location details, from the regions they departed, to the continent aggregation level. This can help analyse details that may not be noticeable on a regional level, but maybe more visible in a continental level.

Dim_Nationality, Dim_Airport and Dim_Status:

Dim_Nationality, Dim_Airport and Dim_Status do not have concept hierarchies, because they are flat dimensions with no concept hierarchies that do not break down into a lower or more detailed level. Since they only contain one singular descriptive value, we cannot apply drill-down, roll-up or aggregation.

2.4. Fact Table Design

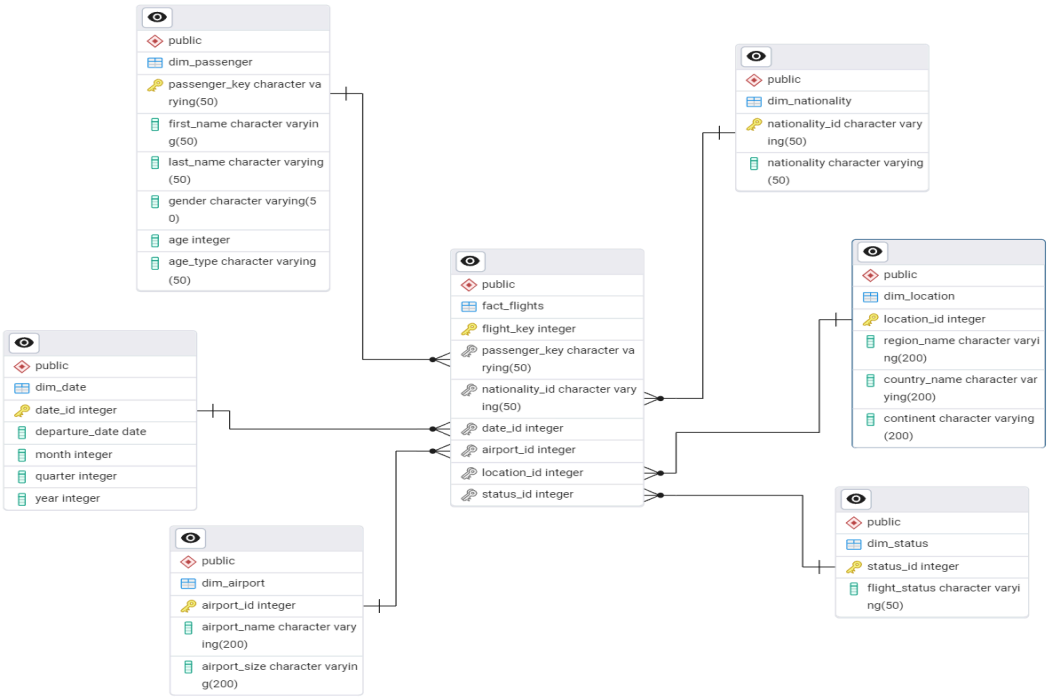
After defining the dimensions, we will now define the fact table. The fact table records a passenger's departure event. Each row represents a passenger traveling from a specific airport from a specific location and on a specific date. Since, the business process and grain are to capture the occurrence of each event, we have created a factless fact table that stores foreign keys to its dimension tables. This will provide a good structure for analysing travel patterns, flight status, and demographics.

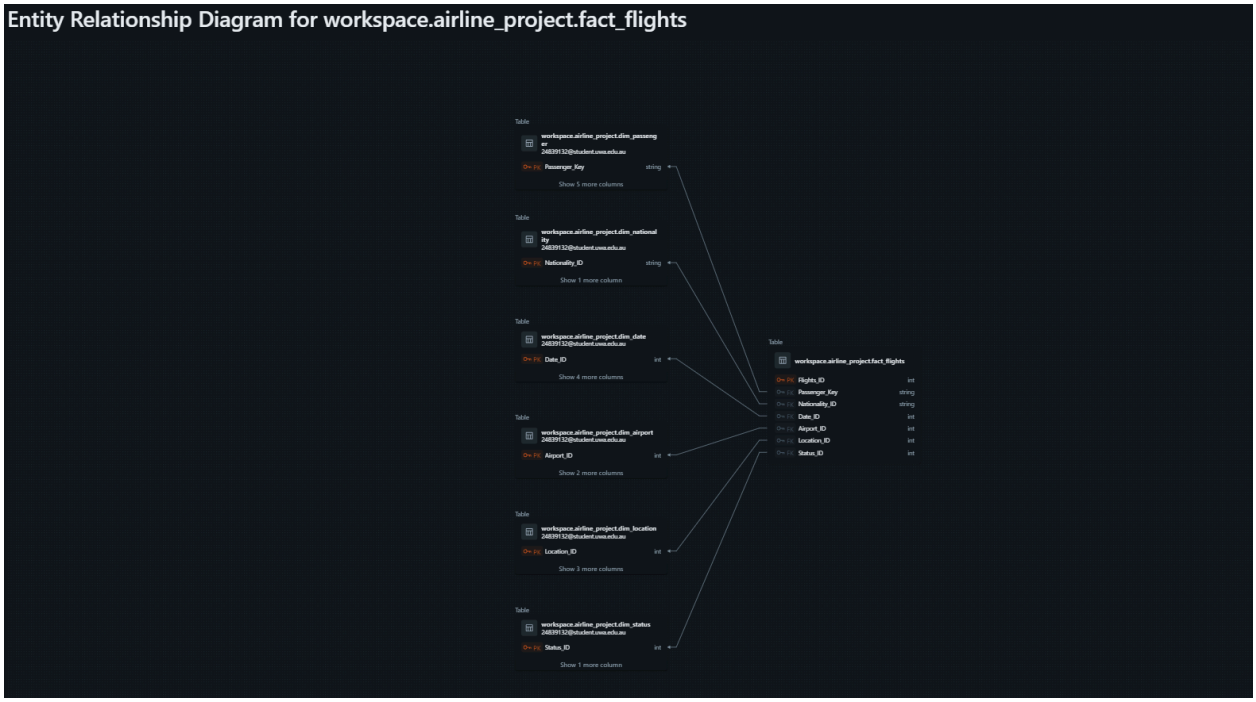
Fact_Flights

Column Name	Description
Flights_ID	The primary key that identifies each flight.
Passenger_Key	The foreign key that references the Dim_Passenger table
Nationality_ID	The foreign key that references the Dim_Nationality table
Date_ID	The foreign key that references the Dim_Date table
Airport_ID	The foreign key that references the Dim_Airport table
Location_ID	The foreign key that references the Dim_Location table
Status_ID	The foreign key that references the Dim_Status table

2.5. Schema and Design Justification

After defining the Dimensions and Fact tables, we have decided to follow a star-schema. This is outlined in the following figures.





Justification for Schema Design

The reason for this star-schema design is because of these principles:

1. Geographic Data Handling

The geographic attributes such as continent, country and region appear repeatedly across many flights and airports. Thus, storing these attributes in Dim_Location prevents duplication and keeps the fact table clean. This structure supports clean and efficient roll-ups and analysis in multiple levels without embedding geographic data inside other dimensions.

2. Storage Optimisation

The other attributes, such as nationality, location, airport size and flight status repeat across many passengers and airports. With normalizing these into separate dimensions (Dim_Nationality, Dim_Location, Dim_Airports and Dim_Flights), the model stores each unique value once rather than repeating it across thousands of records. Thus, reducing storage requirements and keeping the warehouse efficient, especially for large datasets.

3. Data Maintenance

Also, separating these dimensions helps simplify updates. If any label changes, we can update in a single table rather than across multiple dimensions. This should improve consistency and reduce the risk of inconsistent and/or outdated values appearing in the data warehouse.

4. Reduced Redundancy

Again, with repeating attributes for Dim_Nationality, Dim_Location, Dim_Airports and Dim_Flights, keeping these dimensions separated avoids redundancy and ensures these repeated values are not stored multiple times. This improves data quality without complicating the schema.

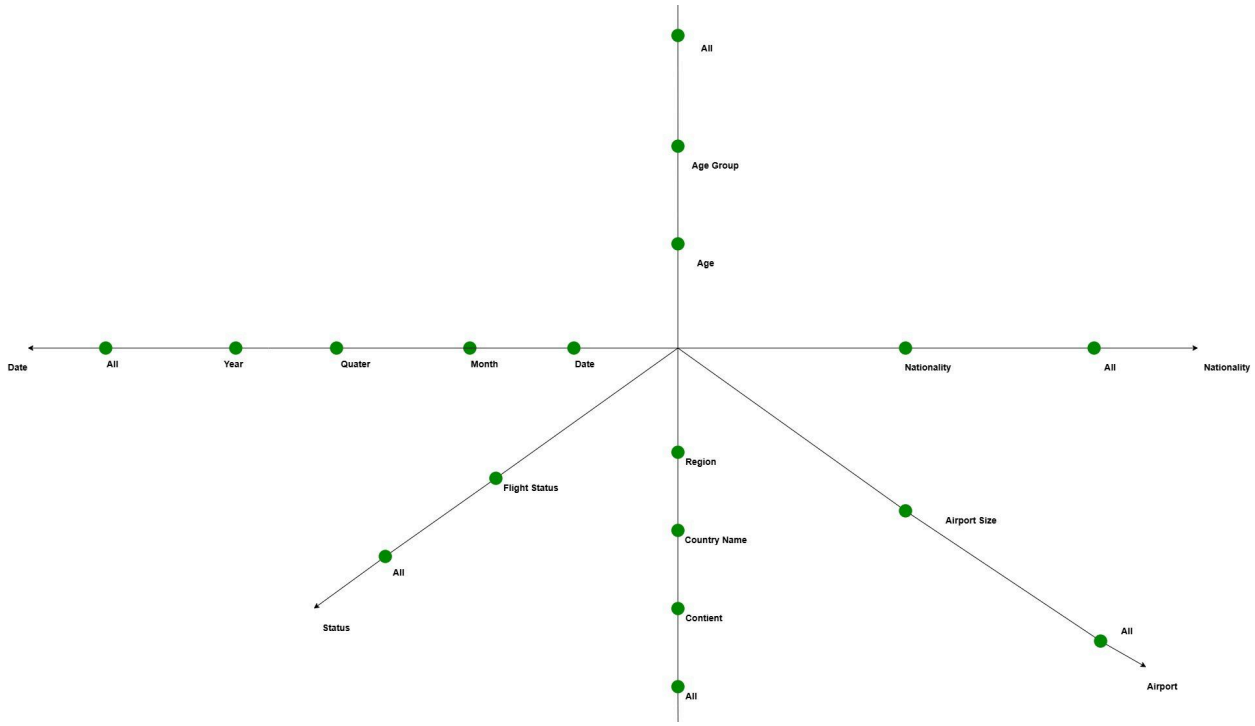
5. Analytical Flexibility

This star-schema structure ensures that all dimensions connect directly to the fact table, enabling fast and intuitive analytical queries. This can help in slicing the data by time, passenger demographics, nationality, airport characteristics, or flight status without the need of applying complex joins. This also supports in analysing a wide range of business questions while keeping the model easy to understand.

This warehouse is simple, intuitive and optimized for analytical queries. The design avoids unnecessary complexity while still supporting a wide range of business queries. This ensures low join complexities, fast query performance and an easy and understandable structure that is user-friendly. Overall, the schema is robust, analytically meaningful, and well-suited to the nature of the dataset and the goal of the warehouse.

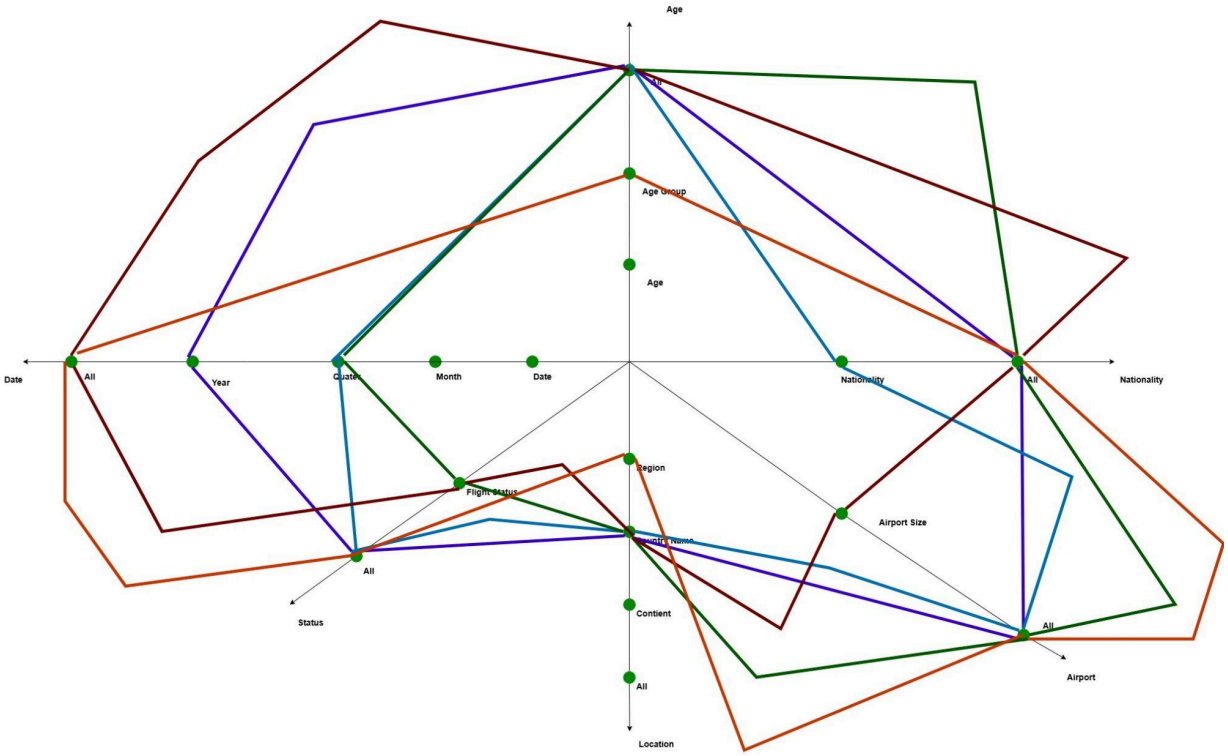
3. StarNet Diagram and Business Queries

3.1. StarNet Diagram



Based on the dimensions we defined and the business queries we developed the star schema (starnet diagram) above.

3.2. Query Footprint



Using the above starnet diagram we defined above we used it map out and create the starnet diagram to answer our business queries.

1. Which countries had the highest air travel demand in 2022. (Top 10)
2. Which nationalities contributed the most to Australia's passenger foot traffic, and in which quarter do they travel most frequently ?
3. Are flight delays and cancellation in Australia correlated with specific quarters?
4. How does airport size (small, medium, large) across Australia influence the flight status.
5. Which age group generates the highest foot traffic across Australian states/ regions ?

4. Data Preparation and ETL process

To prepare the data for dimensional modeling and data mining tasks, we have implemented a structured ETL process using Python on Jupyter Notebooks. This allows us to monitor and validate intermediate results at every step, ensuring transparency and reproducibility. The steps are as followed:

- Extract data into the Python environment and clean all datasets by removing duplicates and trimming white spaces first.
- Rename columns with similar names for each dataset, to make it easier to call without incorrectly using the wrong column name.
- Join datasets and select unique columns to avoid redundancy to structure the data into a clean and analysis ready table
- Recleaned the data by removing null or missing rows.
- Produced dimensions and fact tables for the star schema
- Derived features(e.g. Quarter, Age_Type)
- Exported created dimensions and fact tables into csv files to load into PostgreSQL and Databricks to implement the data warehouse.
- Preparing filtered and enriched subsets of data to support Association Mining affecting flight statuses.

Libraries Setup:

- pandas

We will go through a quick look at the ETL process, for the full ETL please check the Project_1_ETL.ipynb file for the full process.

4.1. Extract

The extraction phase involves reading data from various sources:

- airline Dataset Updated - v2.csv
    - Main dataset, contains the individual passengers departure flight information.
- airports.csv
    - Contains airport information, such as country, location and geographic points.
- countries.csv
    - Contains country level information.
- regions.csv
    - Contains region information for every country it has listed.

To read the data, we import the pandas library and load it into the IDE. We first looked into each dataset by populating the first 3 rows, to verify that the datasets have loaded properly and understand the layout of the datasets.

```
# Importing functions
import pandas as pd
```

```
# reading in the airline datasets
Airline = pd.read_csv('Airline Dataset Updated - v2.csv')
#We first look into the the dataset to get an understanding the variables and the rows.
Airline.head(3)
```

	Passenger ID	First Name	Last Name	Gender	Age	Nationality	Airport Name	Airport Country Code	Country Name	Airport Continent	Continents	Departure Date	Arrival Airport	Pilot Name	Flight Status
0	ABVWIg	Edithe	Leggis	Female	62	Japan	Coldfoot Airport	US	United States	NAM	North America	6/28/2022	CXF	Fransisco Hazeldine	On Time
1	jkXXAX	Elwood	Catt	Male	62	Nicaragua	Kugluktuk Airport	CA	Canada	NAM	North America	12/26/2022	YCO	Marla Parsonage	On Time
2	CdUz2g	Darby	Felgate	Male	67	Russia	Grenoble-Isère Airport	FR	France	EU	Europe	1/18/2022	GNB	Rhonda Amber	On Time

```
# reading in the airport datasets
Airports = pd.read_csv('airports.csv')
#We look into the the dataset to get an understanding the variables and the rows.
Airports.head(3)
```

	id	ident	type	name	latitude_deg	longitude_deg	elevation_ft	continent	iso_country	iso_region	municipality	scheduled_service	icao_code	iata_code	gps_code	local_code	
0	6523	00A	heliport	Total RF Heliport	40.070985	-74.933689	11.0	NaN	US	US-PA	Bensalem	no	NaN	NaN	K00A	00A	https://www.penn
1	323361	00AA	small_airport	Aero B Ranch Airport	38.704022	-101.473911	3435.0	NaN	US	US-KS	Leoti	no	NaN	NaN	00AA	00AA	
2	6524	00AK	small_airport	Lowell Field	59.947733	-151.692524	450.0	NaN	US	US-AK	Anchor Point	no	NaN	NaN	00AK	00AK	

```
# reading in the countries datasets
Countries = pd.read_csv('countries.csv')
#We look into the the dataset to get an understanding the variables and the rows.
Countries.head(3)
```

	id	code	name	continent	wikipedia_link	keywords
0	302672	AD	Andorra	EU	https://en.wikipedia.org/wiki/Andorra	Andorran airports
1	302618	AE	United Arab Emirates	AS	https://en.wikipedia.org/wiki/United_Arab_Emir...	مطارات في الإمارات العربية المتحدة,UAE
2	302619	AF	Afghanistan	AS	https://en.wikipedia.org/wiki/Afghanistan	NaN

[9]:
```
# reading in the regions datasets
Regions = pd.read_csv('regions.csv')
#We look into the the dataset to get an understanding the variables and the rows.
Regions.head(3)
```

[9]:

	id	code	local_code	name	continent	iso_country	wikipedia_link	keywords
0	302811	AD-02	02	Canillo Parish	EU	AD	https://en.wikipedia.org/wiki/Canillo	Airports in Canillo Parish
1	302812	AD-03	03	Encamp Parish	EU	AD	https://en.wikipedia.org/wiki/Encamp	Airports in Encamp Parish
2	302813	AD-04	04	La Massana Parish	EU	AD	https://en.wikipedia.org/wiki/La_Massana	Airports in La Massana Parish

We can see that in the airline dataset, we have 15 variables, and we noticed that airport name and arrival airport are the same, as there was an ETL error when the dataset was encoded, both just record the departure airport in different formats. For now we have considered leaving these two variables untouched, since we will link the main airline dataset with the airport dataset. This is because the airport dataset has some rows missing an iata code but has a corresponding name that we can link to the main dataset. We also removed the country name, since the country code is more useful, because we can link it to the countries dataset.The country and region datasets have a categorical variable related to wikipedia_links and keywords, for our data warehouse, we can consider removing them as they do not provide much information for our business queries. We then checked for missing values, this is to

ensure that we may need to apply imputation to any missing values if needed. We can see that the main airline dataset is not missing any rows, so we may not need to impute the dataset for now.

```
# no missing value found in the main dataset
Airline.isna().sum()
```

```
Passenger ID              0
First Name                0
Last Name                 0
Gender                    0
Age                       0
Nationality               0
Airport Name              0
Airport Country Code      0
Country Name              0
Airport Continent         0
Continents                0
Departure Date            0
Arrival Airport           0
Pilot Name                0
Flight Status             0
dtype: int64
```

We then look into the datatype of the main airline dataset. This is to ensure that each variable is in the correct format so we can load it into Tableau without any problems. Departure Date is in object format, we will convert it into date format in the transformation portion.

```
#We then look into the the dataset to get an understanding the data types for each column
Airline.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 98619 entries, 0 to 98618
Data columns (total 15 columns):
 #   Column                Non-Null Count  Dtype 
---  ------                --------------  ----- 
 0   Passenger ID          98619 non-null  object
 1   First Name            98619 non-null  object
 2   Last Name             98619 non-null  object
 3   Gender                98619 non-null  object
 4   Age                   98619 non-null  int64 
 5   Nationality           98619 non-null  object
 6   Airport Name          98619 non-null  object
 7   Airport Country Code  98619 non-null  object
 8   Country Name          98619 non-null  object
 9   Airport Continent     98619 non-null  object
 10  Continents            98619 non-null  object
 11  Departure Date        98619 non-null  object
 12  Arrival Airport       98619 non-null  object
 13  Pilot Name            98619 non-null  object
 14  Flight Status         98619 non-null  object
dtypes: int64(1), object(14)
memory usage: 11.3+ MB
```

We looked into the datatype and structure of the airport, region and country datasets as well. The datatypes for each variable in each dataset is correct. On the other hand, we noticed that there are some missing variables for each dataset. For the airport dataset, we noticed that some iata codes are missing, but names are not. Similarly, the country and regions datasets are missing some rows for some attributes. We will fix this during the cleaning process.

```
#We then look into the the dataset to get an understanding the data types for each column
#data types are correct, so we dont need to change change the data types
Airports.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 84536 entries, 0 to 84535
Data columns (total 19 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   id                 84536 non-null  int64  
 1   ident              84536 non-null  str    
 2   type               84536 non-null  str    
 3   name               84536 non-null  str    
 4   latitude_deg       84536 non-null  float64
 5   longitude_deg      84536 non-null  float64
 6   elevation_ft       69872 non-null  float64
 7   continent          45083 non-null  str    
 8   iso_country        84244 non-null  str    
 9   iso_region         84536 non-null  str    
 10  municipality       79765 non-null  str    
 11  scheduled_service  84536 non-null  str    
 12  icao_code          9589 non-null   str    
 13  iata_code          9059 non-null   str    
 14  gps_code           43871 non-null  str    
 15  local_code         35918 non-null  str    
 16  home_link          4617 non-null   str    
 17  wikipedia_link     16649 non-null  str    
 18  keywords           20901 non-null  str    
dtypes: float64(3), int64(1), str(15)
memory usage: 12.3 MB
```

```
# airports dataset have missing values, suggesting some airports dont have iata codes
Airports.isnull().sum()
```

```
id                      0
ident                   0
type                    0
name                    0
latitude_deg            0
longitude_deg           0
elevation_ft        14664
continent           39453
iso_country           292
iso_region              0
municipality         4771
scheduled_service       0
icao_code           74947
iata_code           75477
gps_code            40665
local_code          48618
home_link           79919
wikipedia_link      67887
keywords            63635
dtype: int64
```

```
#We then look into the the dataset to get an understanding the data types for each column
#data types are correct, so we dont need to change change the data types
Countries.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 249 entries, 0 to 248
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   id              249 non-null    int64 
 1   code            248 non-null    str   
 2   name            249 non-null    str   
 3   continent       208 non-null    str   
 4   wikipedia_link  249 non-null    str   
 5   keywords        233 non-null    str   
dtypes: int64(1), str(5)
memory usage: 11.8 KB
```

```
# countries dataset have missing values, suggesting some countries dont have continent, keywords encoded into them.
Countries.isnull().sum()
```

```
id                0
code              1
name              0
continent        41
wikipedia_link    0
keywords         16
dtype: int64
```

```
#We then Look into the the dataset to get an understanding the data types for each column
#data types are correct, so we dont need to change change the data types
Regions.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3942 entries, 0 to 3941
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   id              3942 non-null   int64 
 1   code            3942 non-null   str   
 2   local_code      3938 non-null   str   
 3   name            3942 non-null   str   
 4   continent       3502 non-null   str   
 5   iso_country     3927 non-null   str   
 6   wikipedia_link  3673 non-null   str   
 7   keywords        3813 non-null   str   
dtypes: int64(1), str(7)
memory usage: 246.5 KB
```

```
# Regions dataset have missing values, suggesting some Regions dont have continent, keywords and such encoded into them.
Regions.isnull().sum()
```

```
id                0
code              0
local_code        4
name              0
continent       440
iso_country      15
wikipedia_link  269
keywords        129
dtype: int64
```

After looking into the raw structure of the datasets, we then move to the transformation portion, where we clean and preprocess the datasets to ensure consistency and quality before creating the dimension and fact tables.

4.2 Transform

Data Cleaning

We first clean the datasets by trimming and removing white spaces. Next, we delete duplicates so that each row is a unique observation. Lastly, we change the variable names for consistency, since several variables across datasets have similar names. Renaming these variables ensures that we will not encounter any conflicts or ambiguity when joining the datasets. We applied this to the airline, country and region datasets first.

```
# Triming strings, to make it more efficient, first create arracys for all the column names for each data set
airline_col = ['Passenger ID', 'First Name', 'Last Name', 'Gender', 'Nationality', 'Airport Name', 'Airport Country Code', 'Cou
airport_col = ['id', 'ident', 'type', 'name', 'latitude_deg', 'longitude_deg', 'elevation_ft', 'continent', 'iso_country', 'iso
country_col = ['id', 'code', 'name', 'continent', 'wikipedia_link', 'keywords']
region_col = ['id', 'code', 'local_code', 'name', 'continent', 'iso_country', 'wikipedia_link', 'keywords']
```

```
Airline.drop_duplicates()
Airline_cleaned = Airline.copy()
for col in airline_col:
    Airline_cleaned[col] = Airline[col].astype(str).str.strip()

Airline_cleaned = Airline_cleaned.rename(columns={
    'Passenger ID': 'Passenger_ID',
    'First Name': 'First_Name',
    'Last Name': 'Last_Name',
    'Airport Name': 'Airport_Name_1',
    'Airport Country Code': 'Airport_Country_Code',
    'Country Name': 'Country_Name_1',
    'Airport Continent': 'Airport_Continent',
    'Continents': 'Continent_1',
    'Departure Date': 'Departure_Date',
    'Arrival Airport': 'Arrival_Airport',
    'Pilot Name': 'Pilot_Name',
    'Flight Status': 'Flight_Status'
})

Airline_cleaned.shape
```

```
(98619, 15)
```

Joining Datasets

We then combine the four datasets together. First, we combined the the airline dataset with the airport dataset, we first combine them using the ‘Arrival_Airport’ column in airline dataset with the ‘IATA_Code’ in the airport dataset, since iata_codes functions as unique identifiers for

airports and are more reliable when joining the datasets. We then split the ones with matched codes and unmatched codes, this is to make sure we only get the rows with not matching iata codes and to only join them with the airport name. We then filtered the airport names to any duplicates. Lastly, we joined all rows with not matching iata codes with the matching airport names. Then we joined them with the country and region dataset using the airport geographic keys, such as ISO_Region, Regions_Codes, ISO_Country_4 and Country_Code. Then we select the variables we will be using for our database, mainly: Passenger_ID, First_Name, Last_Name, Age, Nationality, Airport_Name_2, Departure_Date, Flight_Status, Airport_Size, Continent_1, Country_Name_3, Region_Name. Finally, we rename the variables to ensure consistency and standardization of naming conventions we will be using in creating dimension and fact tables.

```
# we then join the airport, countries and regions dataset with the airline dataset.
iata_match = Airline_cleaned.merge(Airport_cleaned, how='left', left_on='Arrival_Airport', right_on='IATA_Code')
matched = iata_match[iata_match['IATA_Code'].notna()]
unmatched = iata_match[iata_match['IATA_Code'].isna()].drop(columns=Airport_cleaned.columns)
valid_names = Airport_cleaned['Airport_Name_2'].value_counts()
valid_names = valid_names[valid_names == 1].index
filter_airports = Airport_cleaned[Airport_cleaned['Airport_Name_2'].isin(valid_names)]
name_matched = unmatched.merge(filter_airports, how='left', left_on='Airport_Name_1', right_on='Airport_Name_2')
Join_1 = pd.concat([matched, name_matched], ignore_index=True)
Join_2 = Join_1.merge(Regions_cleaned_v2, how='left', left_on='ISO_Region', right_on='Regions_Code')
Join_3 = Join_2.merge(Countries_cleaned_v2, how='left', left_on='ISO_Country_4', right_on='Country_Code')
```

```
# we then select the main variables we will need for our dimensions and fact tables to answer our business query.
Joined_Dataset = Join_3[['Passenger_ID', 'Gender', 'First_Name', 'Last_Name', 'Age', 'Nationality', 'Airport_Name_2', 'Departure_Date', 'Flight_Status', 'Airport_Size', 'Continent_
```

```
Joined_Dataset.head(3)
```

	Passenger_ID	Gender	First_Name	Last_Name	Age	Nationality	Airport_Name_2	Departure_Date	Flight_Status	Airport_Size	Continent_1	Country_Name_3	Region_Name
0	ABVWIg	Female	Edithe	Leggis	62	Japan	Coldfoot Airport	2022-06-28	On Time	small_airport	North America	United States	Alaska
1	jkXXAX	Male	Elwood	Catt	62	Nicaragua	Kugluktuk Airport	2022-12-26	On Time	small_airport	North America	Canada	Nunavut
2	CdUz2g	Male	Darby	Felgate	67	Russia	Grenoble Alpes Isère Airport	2022-01-18	On Time	medium_airport	Europe	France	Auvergne-Rhône-Alpes

```
Joined_Dataset.head(3)
```

	Passenger_ID	Gender	First_Name	Last_Name	Age	Nationality	Airport_Name_2	Departure_Date	Flight_Status	Airport_Size	Continent_1	Country_Name_3	Region_Name
0	ABVWIg	Female	Edithe	Leggis	62	Japan	Coldfoot Airport	2022-06-28	On Time	small_airport	North America	United States	Alaska
1	jkXXAX	Male	Elwood	Catt	62	Nicaragua	Kugluktuk Airport	2022-12-26	On Time	small_airport	North America	Canada	Nunavut
2	CdUz2g	Male	Darby	Felgate	67	Russia	Grenoble Alpes Isère Airport	2022-01-18	On Time	medium_airport	Europe	France	Auvergne-Rhône-Alpes

We then check for any nulls again. We see that we have 1,667 rows missing in Airport Name, Airport_Size, and Regions Names.

```
# we then check for missing values
Joined_Dataset.isnull().sum()
```

```
Passenger_ID         0
Gender               0
First_Name           0
Last_Name            0
Age                  0
Nationality          0
Airport_Name_2    1667
Departure_Date       0
Flight_Status        0
Airport_Size      1667
Continent_1          0
Country_Name_3       0
Region_Name       1667
dtype: int64
```

This is most likely because the airport dataset does not have a matching iata code or the arrival airport code contained missing/incorrect codes or no matching airport name when we were joining the datasets, thus we were not able to join the airline and airport dataset for some rows. Given we have 98,619, and only 1,667 rows are missing, which is less than 2% of the data, we can remove these null rows to maintain a clean and reliable dataset.

```
# drop rows with missing rows
Joined_data_cleaned = Joined_Dataset.dropna(subset=['Airport_Name_2'])
```

```
# check for null rows
Joined_data_cleaned.isnull().sum()
```

```
Passenger_ID      0
Gender            0
First_Name        0
Last_Name         0
Age               0
Nationality       0
Airport_Name_2    0
Departure_Date    0
Flight_Status     0
Airport_Size      0
Continent_1       0
Country_Name_3    0
Region_Name       0
dtype: int64
```

We then generate Age_Type from the joined dataset, Age_Type was derived from passenger age and groups these passengers into Child, Young_Adult, Adult and Seniors.

```
# function used to create new variable, Age_Type, in passenger dimension table
def age_range(age):
    if age <= 17:
        return 'Child'
    elif (18 <= age) and (age <= 35):
        return 'Young_Adult'
    elif (36 <= age) and (age <= 59):
        return 'Adult'
    else:
        return 'Senior'
# create an Age_Type attribute in the dataset
Final_Dataset['Age_Type'] = Final_Dataset['Age'].apply(age_range)
# show the first 4 rows of the dataset
Final_Dataset.head(4)
```

	Passenger_ID	Gender	First_Name	Last_Name	Age	Nationality	Airport_Name	Departure_Date	Flight_Status	Airport_Size	Continent	Country_Name	Region_Name	Age_Type
0	ABVWIg	Female	Edithe	Leggis	62	Japan	Coldfoot Airport	2022-06-28	On Time	small_airport	North America	United States	Alaska	Senior
1	jkXXAX	Male	Elwood	Catt	62	Nicaragua	Kugluktuk Airport	2022-12-26	On Time	small_airport	North America	Canada	Nunavut	Senior
2	CdUz2g	Male	Darby	Felgate	67	Russia	Grenoble Alpes Isère Airport	2022-01-18	On Time	medium_airport	Europe	France	Auvergne-Rhône-Alpes	Senior
3	BRS38V	Female	Dominica	Pyle	71	China	Ottawa / Gatineau Airport	2022-09-16	Delayed	medium_airport	North America	Canada	Quebec	Senior

We finalized our data set and exported it as a .csv file, where this exported file will serve as a foundation for both transformation and association mining process for our data warehousing project.

```
# check the first 4 rows of the dataset
Final_Dataset.head(4)
```

ge	Nationality	Airport_Name	Departure_Date	Flight_Status	Airport_Size	Continent	Country_Name	Region_Name	Age_Type	Seasons
62	Japan	Coldfoot Airport	2022-06-28	On Time	small_airport	North America	United States	Alaska	Senior	Winter
62	Nicaragua	Kugluktuk Airport	2022-12-26	On Time	small_airport	North America	Canada	Nunavut	Senior	Summer
67	Russia	Grenoble Alpes Isère Airport	2022-01-18	On Time	medium_airport	Europe	France	Auvergne-Rhône-Alpes	Senior	Summer
71	China	Ottawa / Gatineau Airport	2022-09-16	Delayed	medium_airport	North America	Canada	Quebec	Senior	Spring

4.3 Load

We first create our Dim_Passenger dataset, we first create the unique identifier, Passenger_Key, to help identify each unique passenger by using First_Name, Last_Name, Gender, Age, then Age_Type.

```
# passenger_df, removing duplicates
passenger_df = Final_Dataset[['Passenger_ID', 'First_Name', 'Last_Name', 'Gender', 'Age', 'Age_Type']].drop_duplicates()
# adding an id/key
p_index = range(101, 101+len(passenger_df))
passenger_df['Passenger_Key'] = ['A' + str(i) for i in p_index]
# rearrange the columns
passenger_df = passenger_df[['Passenger_Key','First_Name', 'Last_Name', 'Gender', 'Age', 'Age_Type']]
passenger_df.head()
```

	Passenger_Key	First_Name	Last_Name	Gender	Age	Age_Type
0	A101	Edithe	Leggis	Female	62	Senior
1	A102	Elwood	Catt	Male	62	Senior
2	A103	Darby	Felgate	Male	67	Senior
3	A104	Dominica	Pyle	Female	71	Senior
4	A105	Bay	Pencost	Male	21	Young_Adult

Next, We create the Dim_Nationality dimension, we create the unique identifier for nationality by getting each unique nationality in the dataset. This contains the Nationality_ID and Nationality of that specific ID.

```
# nationality_df, removing duplicates
nationality_df = Final_Dataset[['Nationality']].drop_duplicates()
# adding an id/key
n_index = range(101, 101+len(nationality_df))
nationality_df['Nationality_ID'] = ['N' + str(i) for i in n_index]
# rearrange the columns
nationality_df = nationality_df[['Nationality_ID', 'Nationality']]
nationality_df.head()
```

	Nationality_ID	Nationality
0	N101	Japan
1	N102	Nicaragua
2	N103	Russia
3	N104	China
5	N105	Brazil

We then create the Dim_Date dimension, we create the unique identifier for date by getting each unique Departure_Date in the dataset. This Dimension includes Date_ID, Departure_Date, Month, Quarter, Year.

```
# creating a date dimension table
date_df = Final_Dataset[['Departure_Date', 'Seasons']].drop_duplicates()
date_df['Month'] = pd.to_datetime(date_df['Departure_Date'], dayfirst = False).dt.month
date_df['Quarter'] = pd.to_datetime(date_df['Departure_Date'], dayfirst = False).dt.quarter
date_df['Year'] = pd.to_datetime(date_df['Departure_Date'], dayfirst = False).dt.year
date_df['Date_ID'] = range(1, len(date_df)+1)
date_df = date_df[['Date_ID', 'Departure_Date', 'Month', 'Quarter','Year', 'Seasons']]
date_df.head(4)
```

	Date_ID	Departure_Date	Month	Quarter	Year	Seasons
0	1	2022-06-28	6	2	2022	Winter
1	2	2022-12-26	12	4	2022	Summer
2	3	2022-01-18	1	1	2022	Summer
3	4	2022-09-16	9	3	2022	Spring

We then create the Dim_Location dimension, we create the unique identifier for location by getting each unique Region_Name, Country_Name, Continent in the dataset. This Dimension includes Location_ID, Region_Name, Country_Name, Continent.

```
location_df = Final_Dataset[['Region_Name', 'Country_Name', 'Continent']].drop_duplicates()
location_df['Location_ID'] = range(1, len(location_df)+1)
location_df = location_df[['Location_ID', 'Region_Name', 'Country_Name', 'Continent']]
location_df.head()
```

	Location_ID	Region_Name	Country_Name	Continent
0	1	Alaska	United States	North America
1	2	Nunavut	Canada	North America
2	3	Auvergne-Rhône-Alpes	France	Europe
3	4	Quebec	Canada	North America
4	5	California	United States	North America

We then create the Dim_Airport dimension, we create the unique identifier for the airport by getting each unique airport name and airport size in the dataset. This Dimension includes Airport_ID, Airport_Name, and Airport_Size.

```
# airport_df, removing duplicates
airport_df = Final_Dataset[['Airport_Name', 'Airport_Size']].drop_duplicates()
# adding an id/key
airport_df['Airport_ID'] = range(1, len(airport_df)+1)
# rearrange the columns
airport_df = airport_df[['Airport_ID', 'Airport_Name', 'Airport_Size']]
airport_df.head()
```

	Airport_ID	Airport_Name	Airport_Size
0	1	Coldfoot Airport	small_airport
1	2	Kugluktuk Airport	small_airport
2	3	Grenoble Alpes Isère Airport	medium_airport
3	4	Ottawa / Gatineau Airport	medium_airport
4	5	Gillespie Field	small_airport

We then create the Dim_Status dimension, we create the unique identifier for flight status by getting each unique flight status in the dataset. This Dimension includes Status_ID, and Status.

```
# airport_df, removing duplicates
status_df = Final_Dataset[['Flight_Status']].drop_duplicates()
# adding an id/key
status_df['Status_ID'] = range(1, len(status_df)+1)
# rearrange the columns
status_df = status_df[['Status_ID', 'Flight_Status']]
status_df.head()
```

	Status_ID	Flight_Status
0	1	On Time
3	2	Delayed
7	3	Cancelled

Lastly, we create the factless fact table, we join the cleaned staging dataset to each dimension table, we do this by merging the dataset using the descriptions of each dimension and then getting the main fact table by selecting the primary keys of the dimension tables and they form the foreign keys for the fact table, then we create a surrogate primary key for the fact table to uniquely track each passenger-flight event. We then export all the dimension and fact tables so we can use them in creating our star-schema data warehouse.

```
#fact table
dfs = [passenger_df, nationality_df, date_df, airport_df, location_df, status_df]
flights_df = Final_Dataset.copy()
flights_df = flights_df.merge(passenger_df, on = ['First_Name','Last_Name', 'Age']).merge(nationality_df, on = ['Nationality']).merge(date_df, on=['Departure_Date']).merge(airp
flights_df

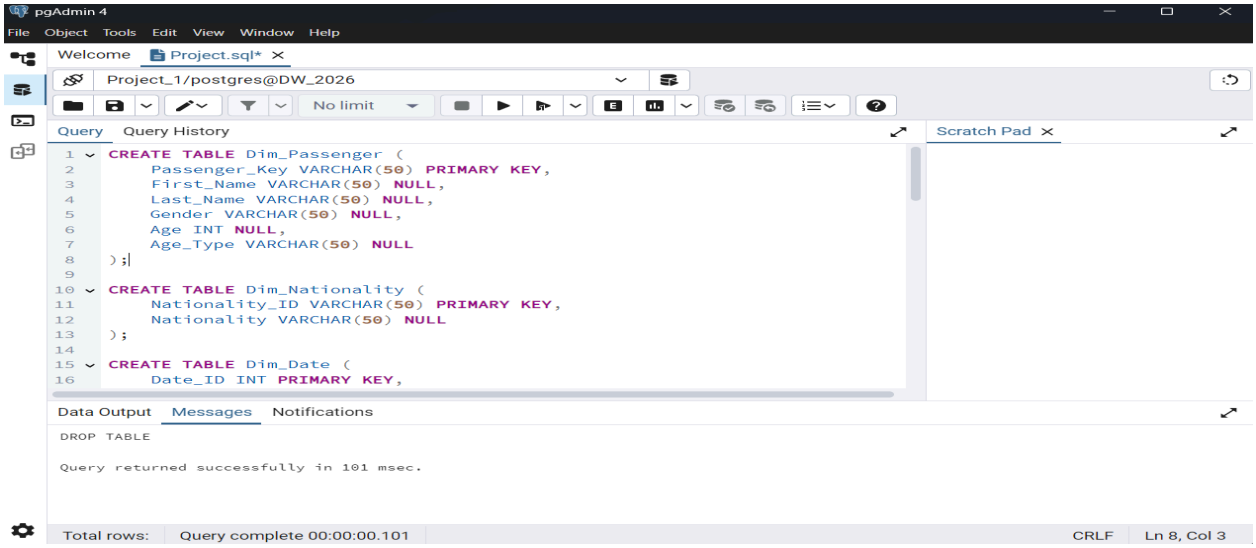
flights_df['Flights_ID'] = range(1, len(flights_df)+1)
flights_df = flights_df[['Flights_ID', 'Passenger_Key', 'Nationality_ID', 'Date_ID', 'Airport_ID', 'Location_ID', 'Status_ID']]
flights_df.head()
```

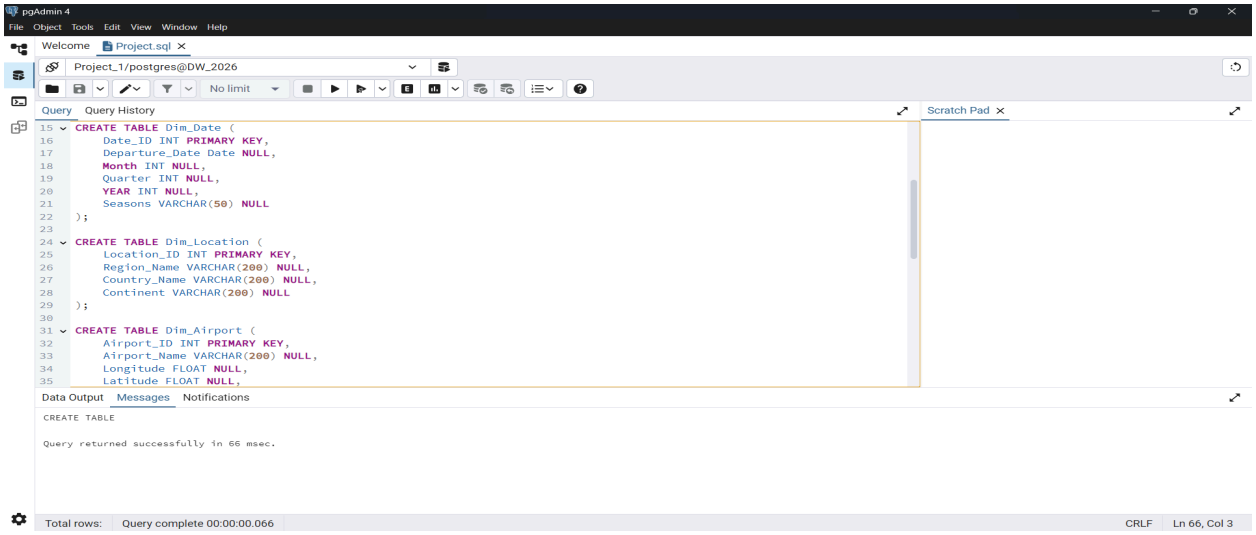
	Flights_ID	Passenger_Key	Nationality_ID	Date_ID	Airport_ID	Location_ID	Status_ID
0	1	A101	N101	1	1	1	1
1	2	A88590	N135	347	1	1	1
2	3	A85678	N106	158	1	1	1
3	4	A2456	N112	349	1	1	1
4	5	A52190	N147	75	1	1	1

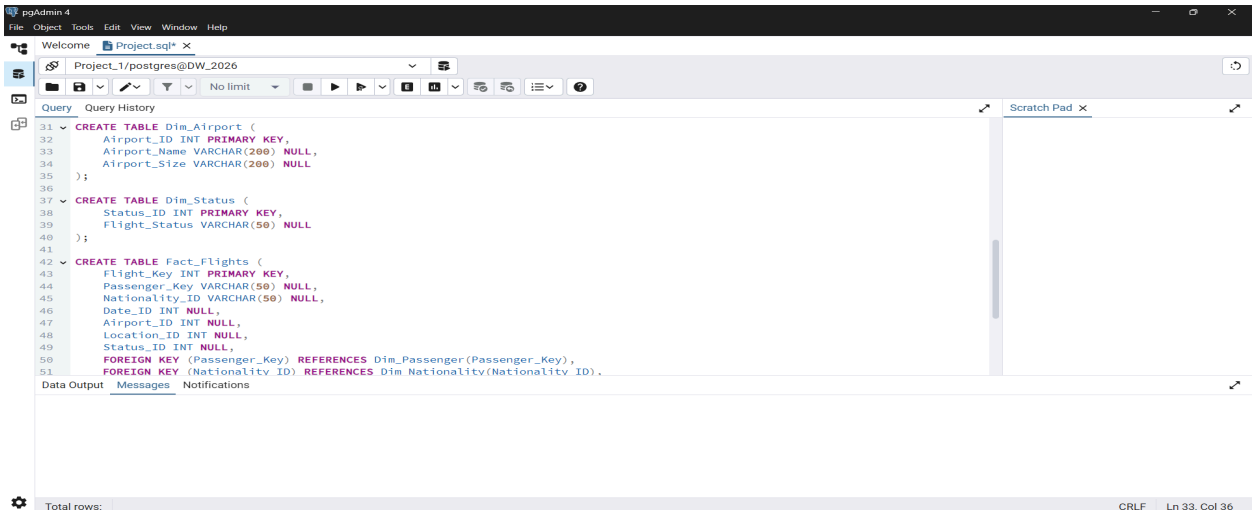
```
# converting to csv
csv_name = ['Dim_Passenger.csv','Dim_Nationality.csv', 'Dim_Date.csv', 'Dim_Airport.csv', 'Dim_Location.csv', 'Dim_Status.csv', 'Fact_Flights.csv']
dfs.append(flights_df)
for i in range(len(csv_name)):
    dfs[i].to_csv(csv_name[i], index = False, quoting=1, encoding='utf-8')
```

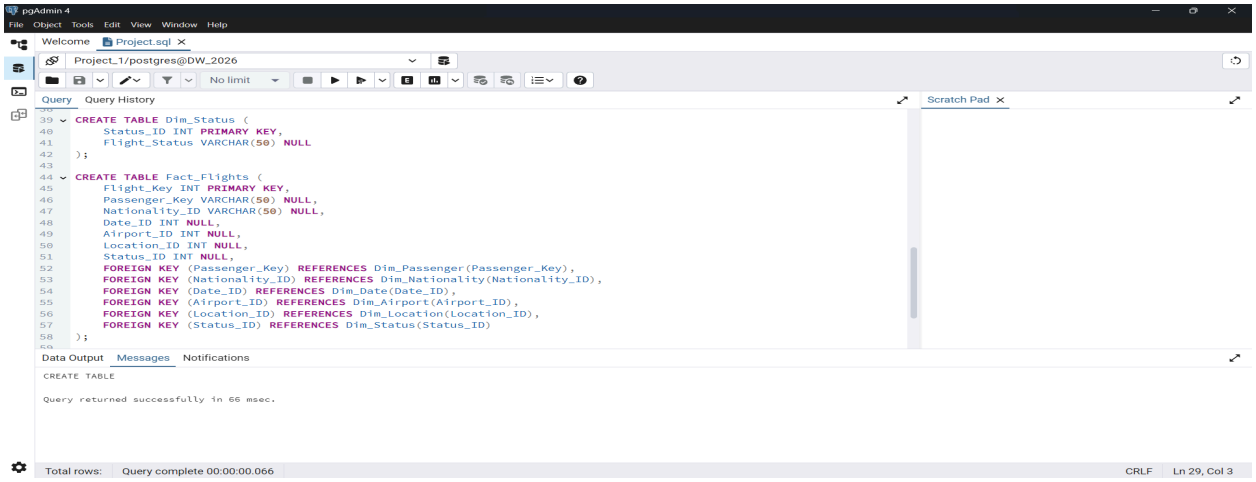
4.4. Data Warehouse Implementation

For this project, we create the data warehouse by using both PostgreSQL and Databricks. For PostgreSQL we first create tables for the dimension and fact tables. Each table was defined with a surrogate primary key, data type for each variable and foreign key references for the fact table to ensure referential integrity.

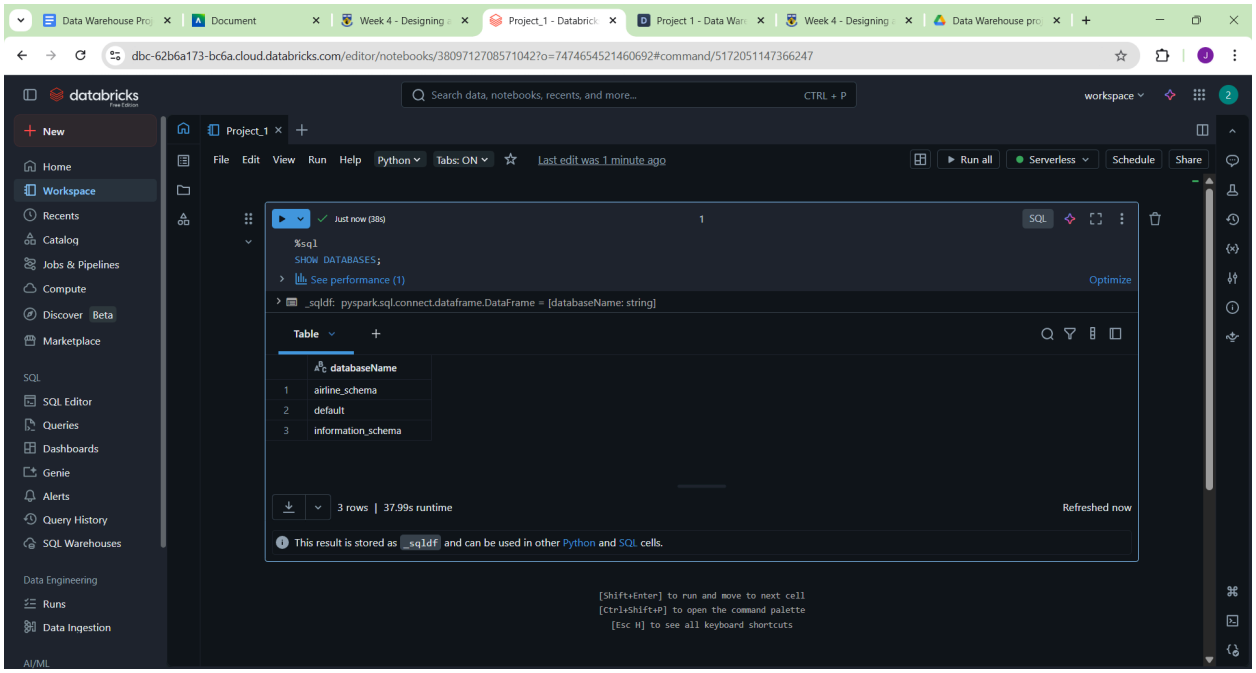

pgAdmin 4
File Object Tools Edit View Window Help
Welcome
Project.sql*
Project_1/postgres@DW_2026
No limit
Query
Query History
Scratch Pad
CREATE TABLE Dim_Passenger (
Passenger_Key VARCHAR(50) PRIMARY KEY,
First_Name VARCHAR(50) NULL,
Last_Name VARCHAR(50) NULL,
Gender VARCHAR(50) NULL,
Age INT NULL,
Age_Type VARCHAR(50) NULL
);
CREATE TABLE Dim_Nationality (
Nationality_ID VARCHAR(50) PRIMARY KEY,
Nationality VARCHAR(50) NULL
);
CREATE TABLE Dim_Date (
Date_ID INT PRIMARY KEY,
Data Output
Messages
Notifications
DROP TABLE
Query returned successfully in 101 msec.
Total rows:
Query complete 00:00:00.101
CRLF
Ln 8, Col 3

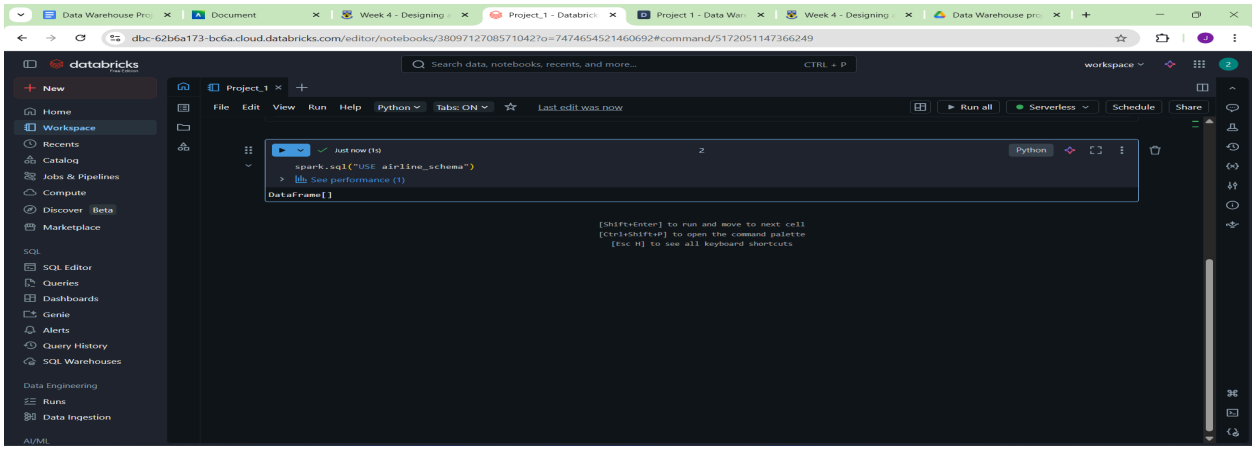

pgAdmin 4
File Object Tools Edit View Window Help
Welcome
Project.sql
Project_1/postgres@DW_2026
No limit
Query
Query History
Scratch Pad
CREATE TABLE Dim_Date (
Date_ID INT PRIMARY KEY,
Departure_Date Date NULL,
Month INT NULL,
Quarter INT NULL,
YEAR INT NULL,
Seasons VARCHAR(50) NULL
);
CREATE TABLE Dim_Location (
Location_ID INT PRIMARY KEY,
Region_Name VARCHAR(200) NULL,
Country_Name VARCHAR(200) NULL,
Continent VARCHAR(200) NULL
);
CREATE TABLE Dim_Airport (
Airport_ID INT PRIMARY KEY,
Airport_Name VARCHAR(200) NULL,
Longitude FLOAT NULL,
Latitude FLOAT NULL,
Data Output
Messages
Notifications
CREATE TABLE
Query returned successfully in 66 msec.
Total rows:
Query complete 00:00:00.066
CRLF
Ln 66, Col 3


pgAdmin 4
File Object Tools Edit View Window Help
Welcome
Project.sql*
Project_1/postgres@DW_2026
No limit
Query
Query History
Scratch Pad
CREATE TABLE Dim_Airport (
Airport_ID INT PRIMARY KEY,
Airport_Name VARCHAR(200) NULL,
Airport_Size VARCHAR(200) NULL
);
CREATE TABLE Dim_Status (
Status_ID INT PRIMARY KEY,
Flight_Status VARCHAR(50) NULL
);
CREATE TABLE Fact_Flights (
Flight_Key INT PRIMARY KEY,
Passenger_Key VARCHAR(50) NULL,
Nationality_ID VARCHAR(50) NULL,
Date_ID INT NULL,
Airport_ID INT NULL,
Location_ID INT NULL,
Status_ID INT NULL,
FOREIGN KEY (Passenger_Key) REFERENCES Dim_Passenger(Passenger_Key),
FOREIGN KEY (Nationality_ID) REFERENCES Dim_Nationality(Nationality_ID),
Data Output
Messages
Notifications
Total rows:
CRLF
Ln 33, Col 36



For Databricks, we first import sql library into the databricks IDE, then go to the working directory where we have loaded our cleaned datasets. We then create tables for the dimension and fact tables similar to the PostgreSQL process when creating our tables.






```sql
%sql
CREATE TABLE airline_project.Dim_Passenger (
    Passenger_Key STRING,
    First_Name STRING,
      Last_Name STRING,
      Gender STRING,
      Age INT,
      Age_Type STRING,
    CONSTRAINT pk_dimpassenger PRIMARY KEY(Passenger_Key)
);

CREATE TABLE airline_project.Dim_Nationality (
    Nationality_ID STRING,
    Nationality STRING,
    CONSTRAINT pk_dimnationality PRIMARY KEY(Nationality_ID)
);

CREATE TABLE airline_project.Dim_Date (
    Date_ID INT,
    Departure_Date Date,
      Month INT,
      Quarter INT,
      YEAR INT,
    CONSTRAINT pk_dimdate PRIMARY KEY(Date_ID)
);

```

```sql
CREATE TABLE Dim_Location (
    Location_ID INT,
    Region_Name STRING,
      Country_Name STRING,
      Continent STRING,
    CONSTRAINT pk_dimlocation PRIMARY KEY(Location_ID)
);

CREATE TABLE Dim_Airport (
    Airport_ID INT,
    Airport_Name STRING,
      Airport_Size STRING,
    CONSTRAINT pk_dimairport PRIMARY KEY(Airport_ID)
);

CREATE TABLE Dim_Status (
    Status_ID INT,
    Flight_Status STRING,
    CONSTRAINT pk_dimstatus PRIMARY KEY(Status_ID)
);

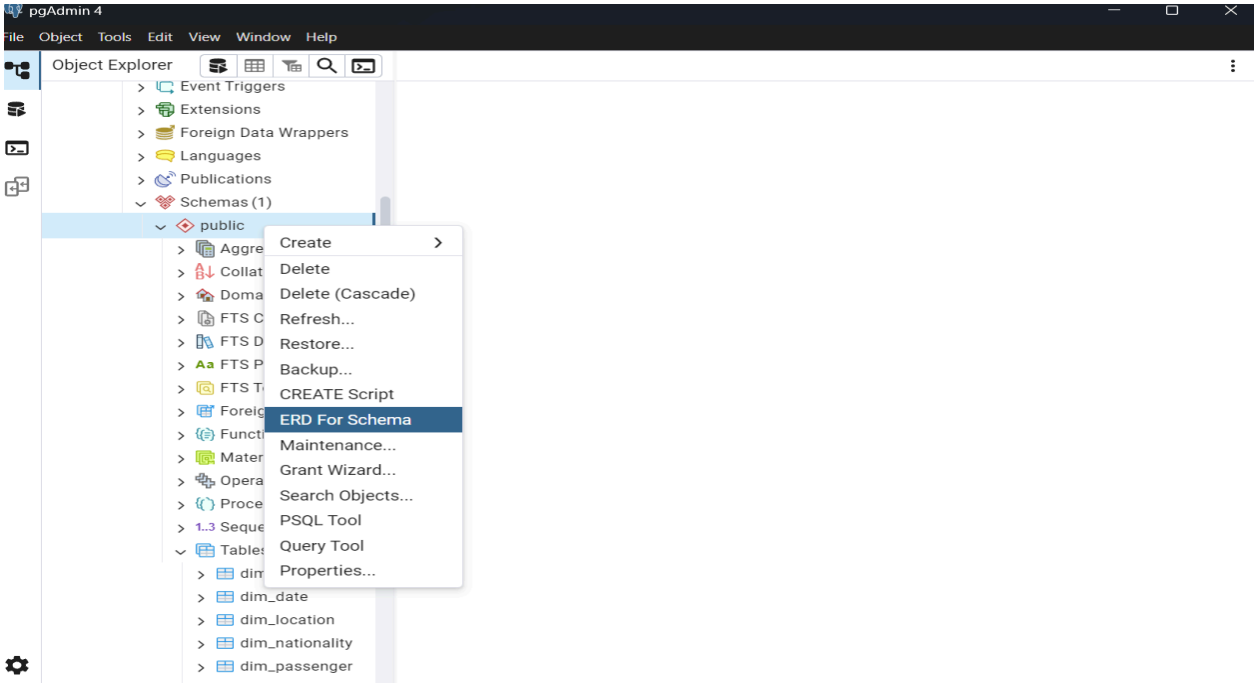

CREATE TABLE Fact_Flights (
    Flight_Key INT,
    Passenger_Key STRING,
    Nationality_ID STRING,
    Date_ID INT
```

```
    Airport_Size STRING,
    CONSTRAINT pk_dimairport PRIMARY KEY(Airport_ID)
);

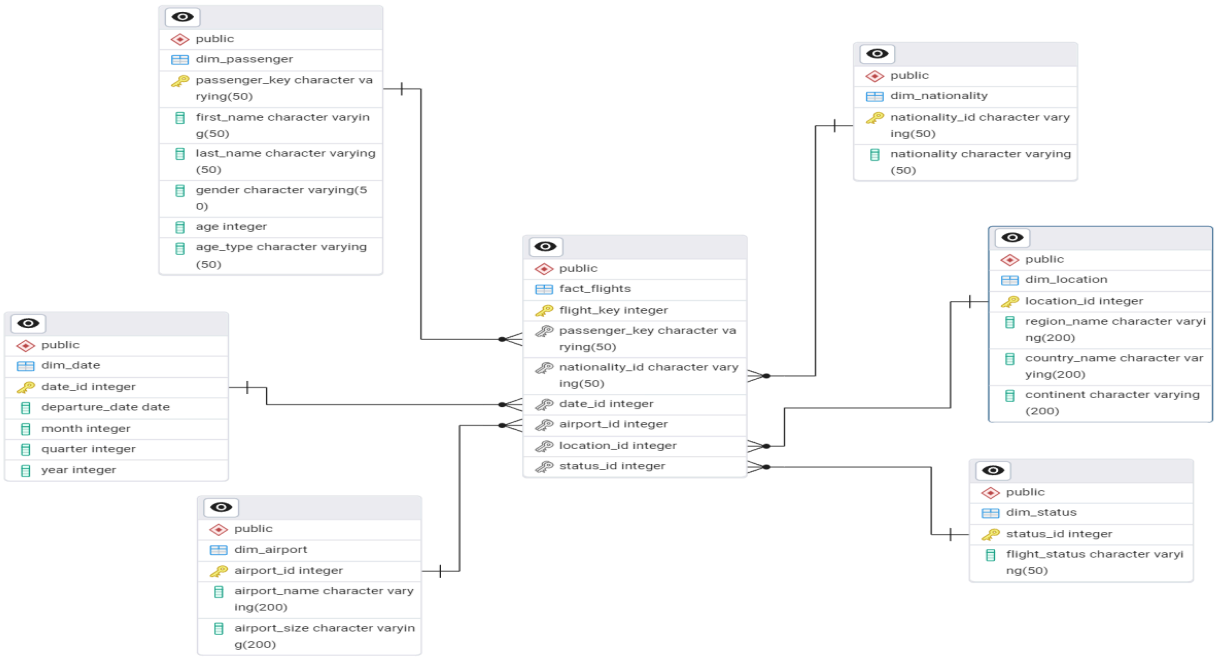
CREATE TABLE Dim_Status (
    Status_ID INT,
    Flight_Status STRING,
    CONSTRAINT pk_dimstatus PRIMARY KEY(Status_ID)
);


CREATE TABLE Fact_Flights (
    Flight_Key INT,
    Passenger_Key STRING,
    Nationality_ID STRING,
    Date_ID INT,
    Airport_ID INT,
    Location_ID INT,
    Status_ID INT,
    CONSTRAINT pk_dimfactflights PRIMARY KEY(Flight_Key),
    CONSTRAINT fk_dimpassenger FOREIGN KEY (Passenger_Key) REFERENCES Dim_Passenger(Passenger_Key),
      CONSTRAINT fk_dimnationality FOREIGN KEY (Nationality_ID) REFERENCES Dim_Nationality(Nationality_ID),
      CONSTRAINT fk_dimdate FOREIGN KEY (Date_ID) REFERENCES Dim_Date(Date_ID),
      CONSTRAINT fk_dimairport FOREIGN KEY (Airport_ID) REFERENCES Dim_Airport(Airport_ID),
      CONSTRAINT fk_dimlocation FOREIGN KEY (Location_ID) REFERENCES Dim_Location(Location_ID),
      CONSTRAINT fk_dimstatus FOREIGN KEY (Status_ID) REFERENCES Dim_Status(Status_ID)
);
```

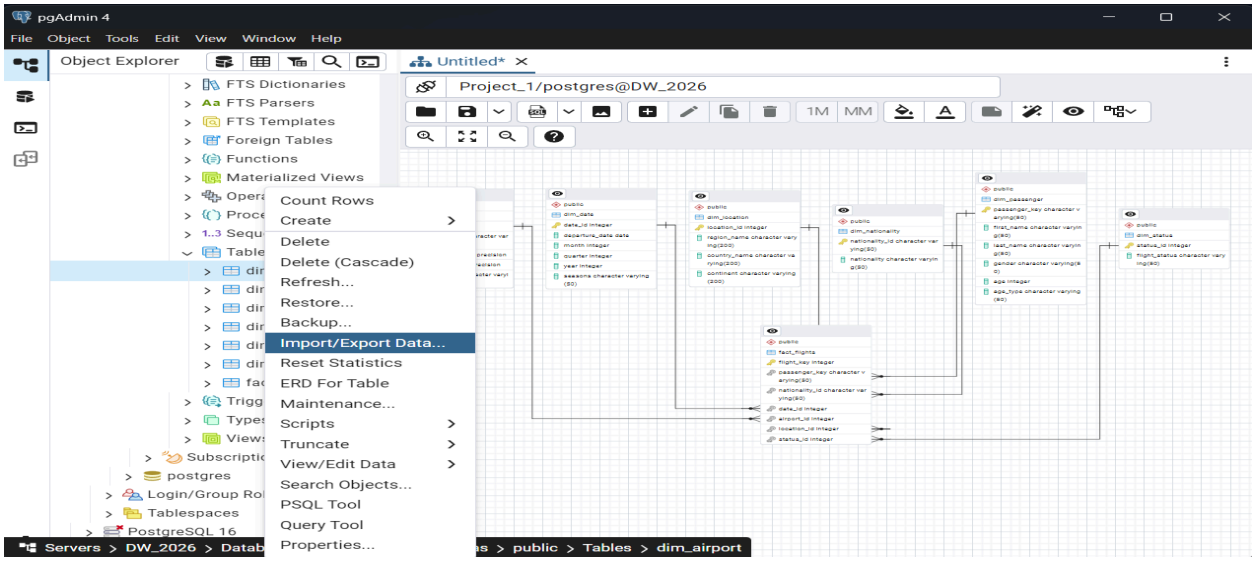
For PostgreSQL, after creating the databases, we first go and create the Star-Schema by clicking on ERD for schema and populating the star schema.

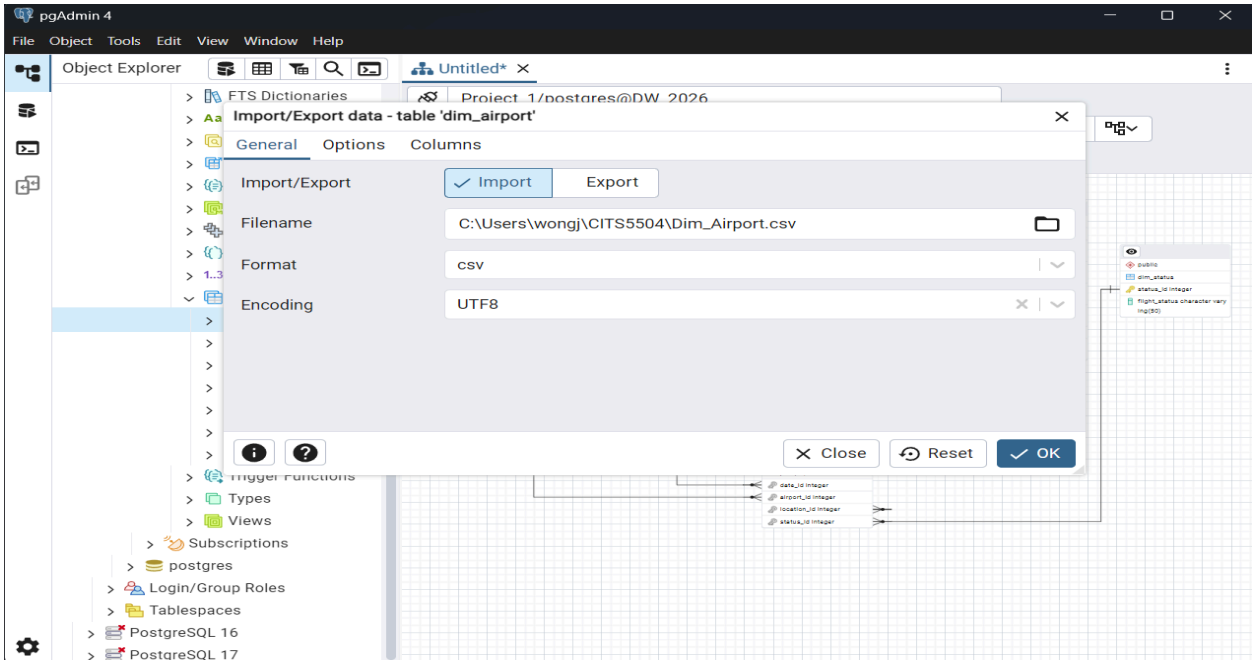


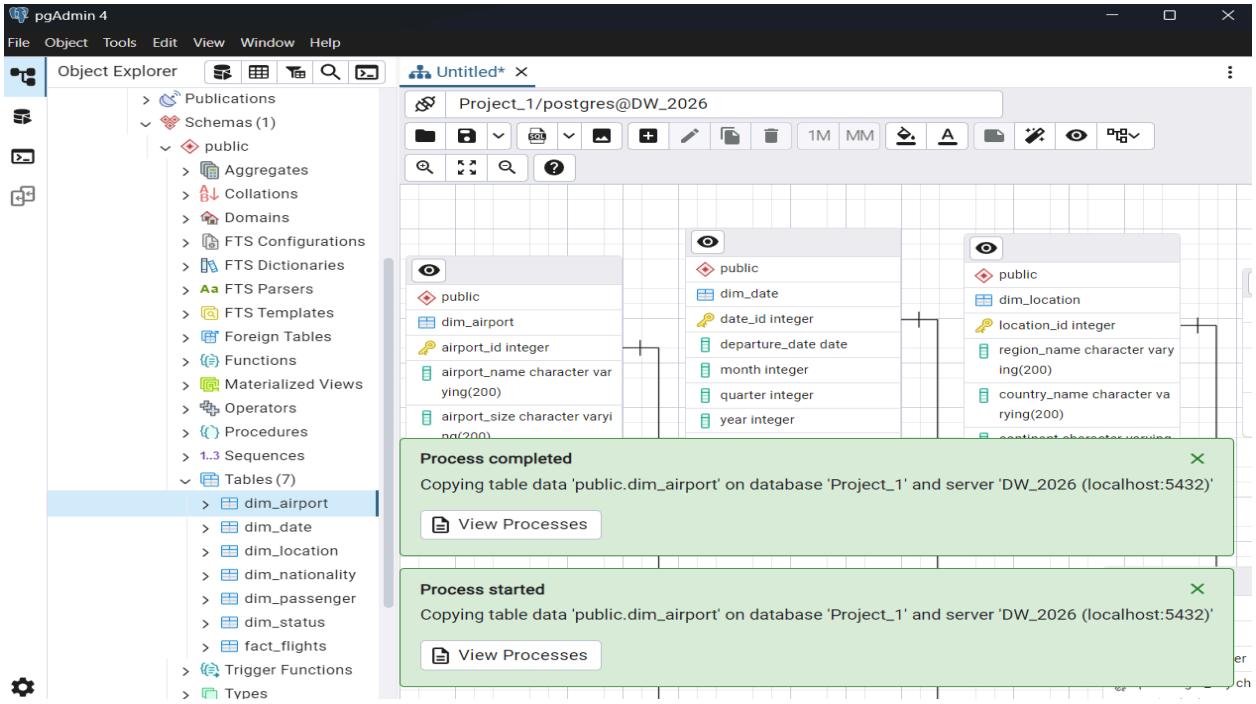
We rearrange the schema and we get a star schema shown below.



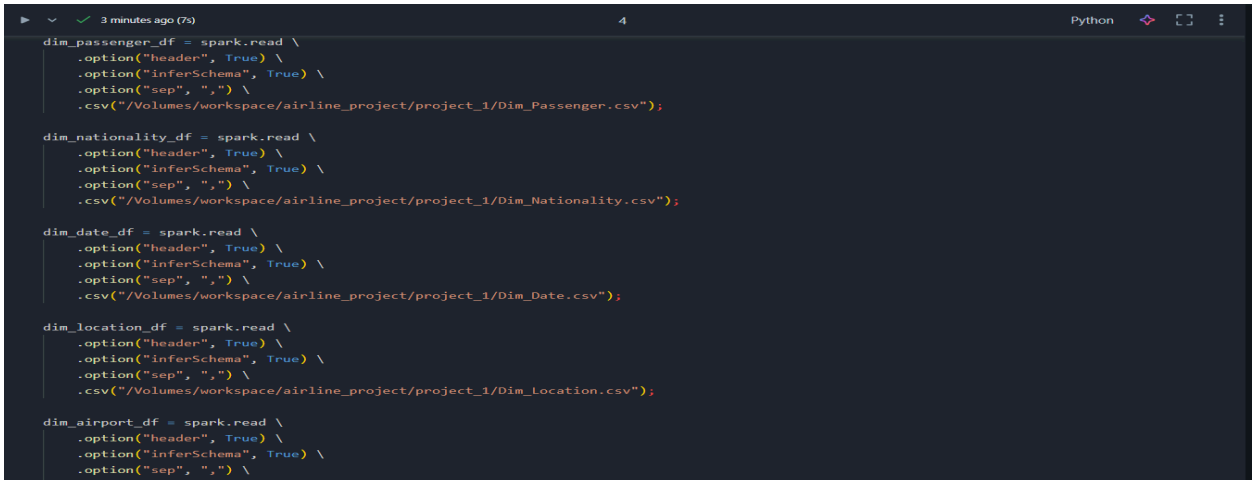
We then manually populate the data into each table, by right clicking on the tables and importing them one by one, as shown below.







For databricks, we load the datasets from the transformation portion and store them into some name values in databricks as shown below.





We then load in the data from the saved name values and store them into the tables created earlier by using the code below. After, we go and view the relationships of the fact table and we have created the ER diagram for the star schema below.

```
▶  ✓ 3 minutes ago (14s)                                                    5

dim_passenger_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Dim_Passenger");

dim_passenger_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Dim_Passenger");

dim_nationality_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Dim_Nationality");

dim_date_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Dim_Date");

dim_location_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Dim_Location");

dim_airport_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Dim_Airport");

dim_status_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Dim_Status");
```

```
▶  ✓ 4 minutes ago (14s)                                                    5

    .mode("append") \
    .saveAsTable("airline_project.Dim_Passenger");

dim_nationality_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Dim_Nationality");

dim_date_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Dim_Date");

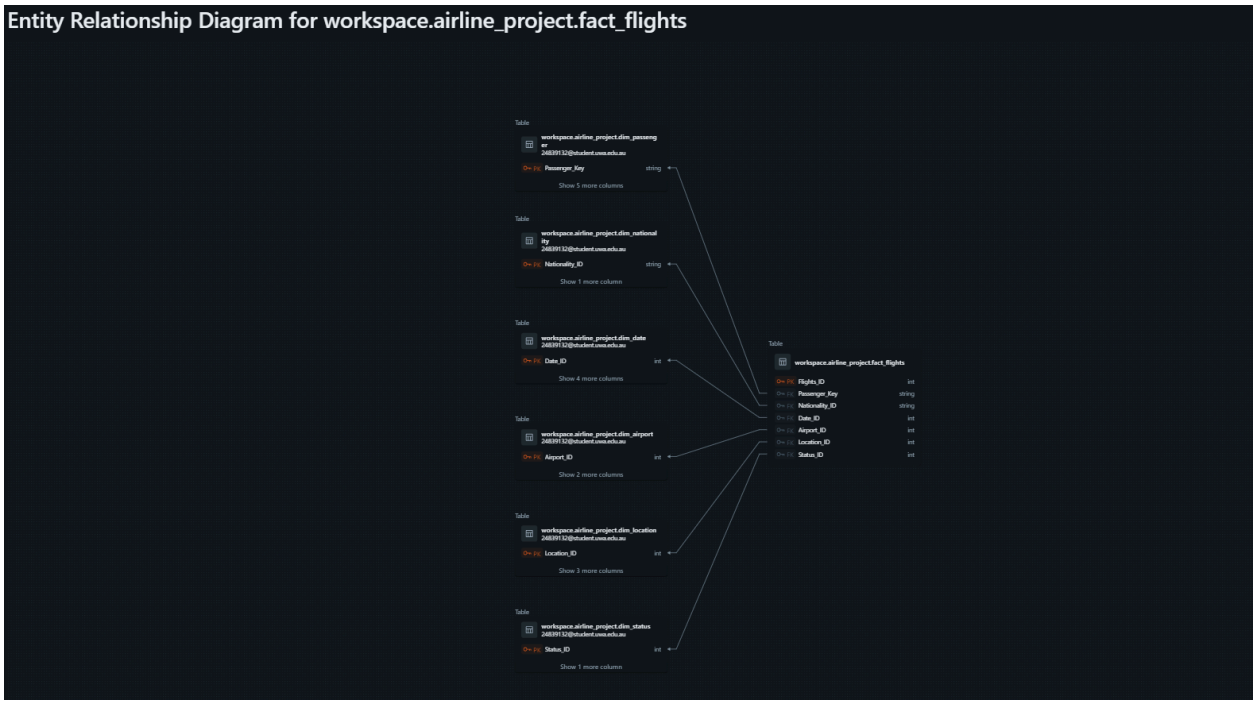
dim_location_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Dim_Location");

dim_airport_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Dim_Airport");

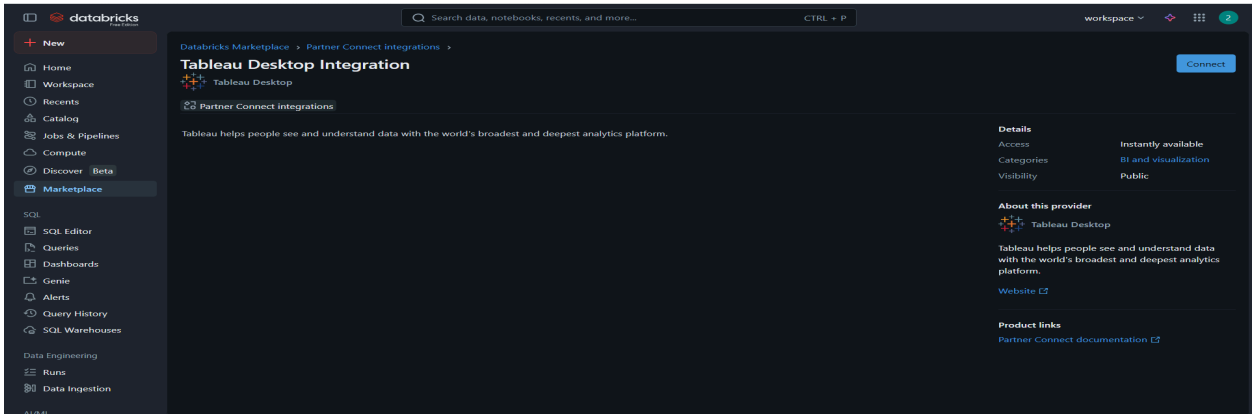
dim_status_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Dim_Status");

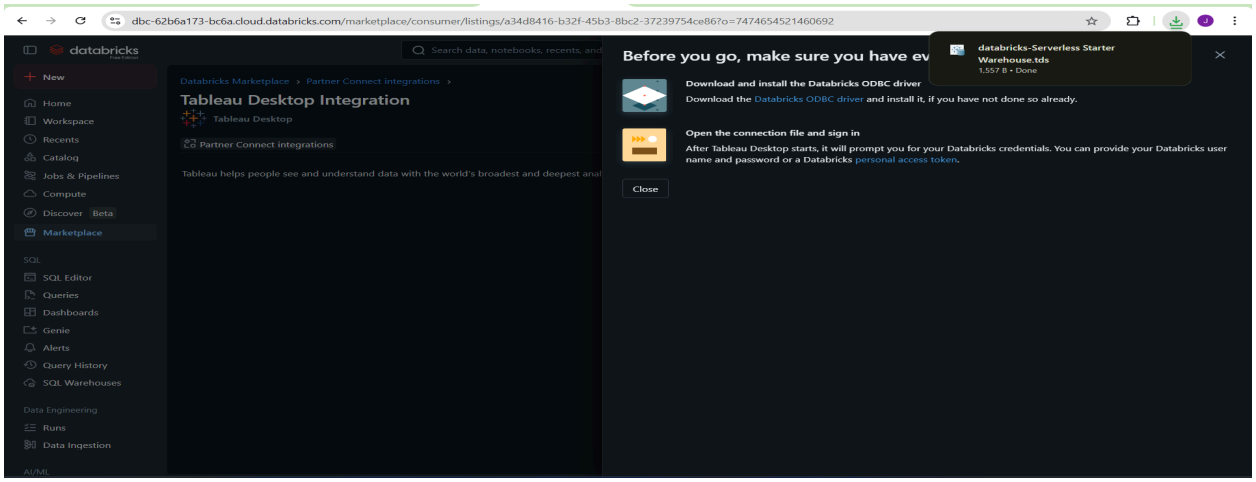
fact_flights_df.write \
    .mode("append") \
    .saveAsTable("airline_project.Fact_Flights");

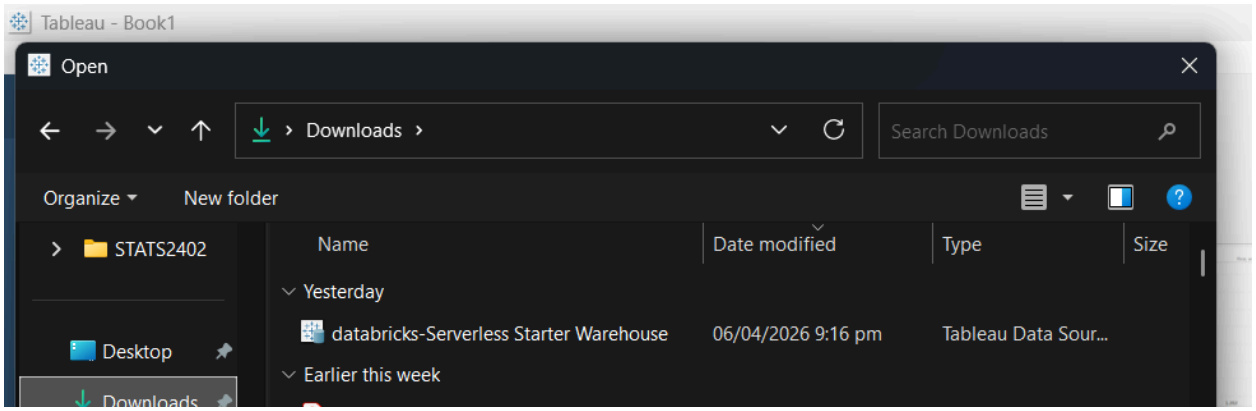
> See performance (8)
```



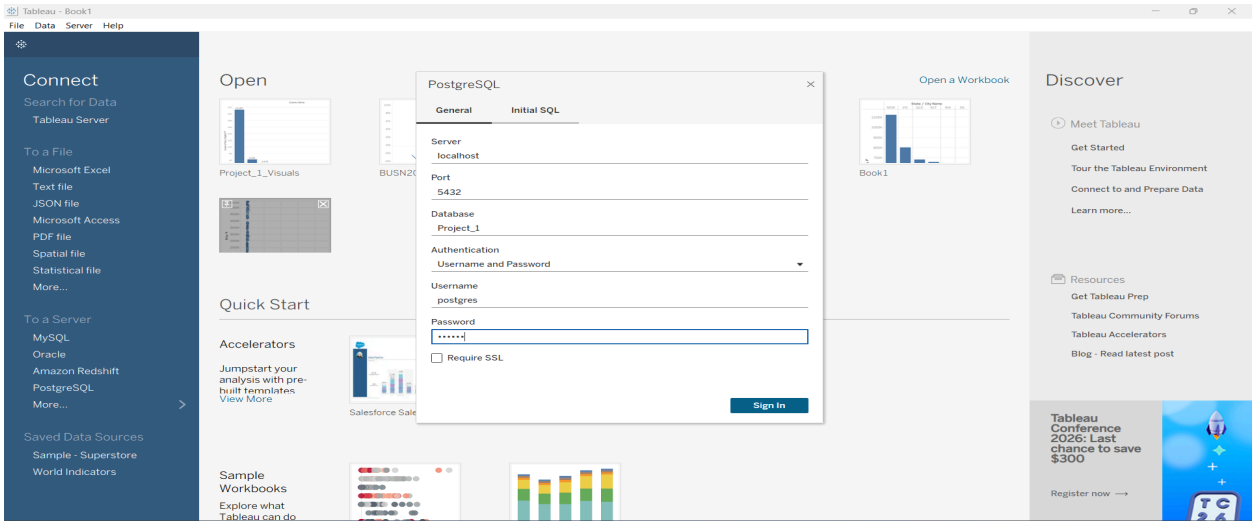
We then load the database into Tableau, which we will use for our visualization of our business queries, we first download the data bricks .tds file and use it to connect Tableau with our databricks data warehouse.



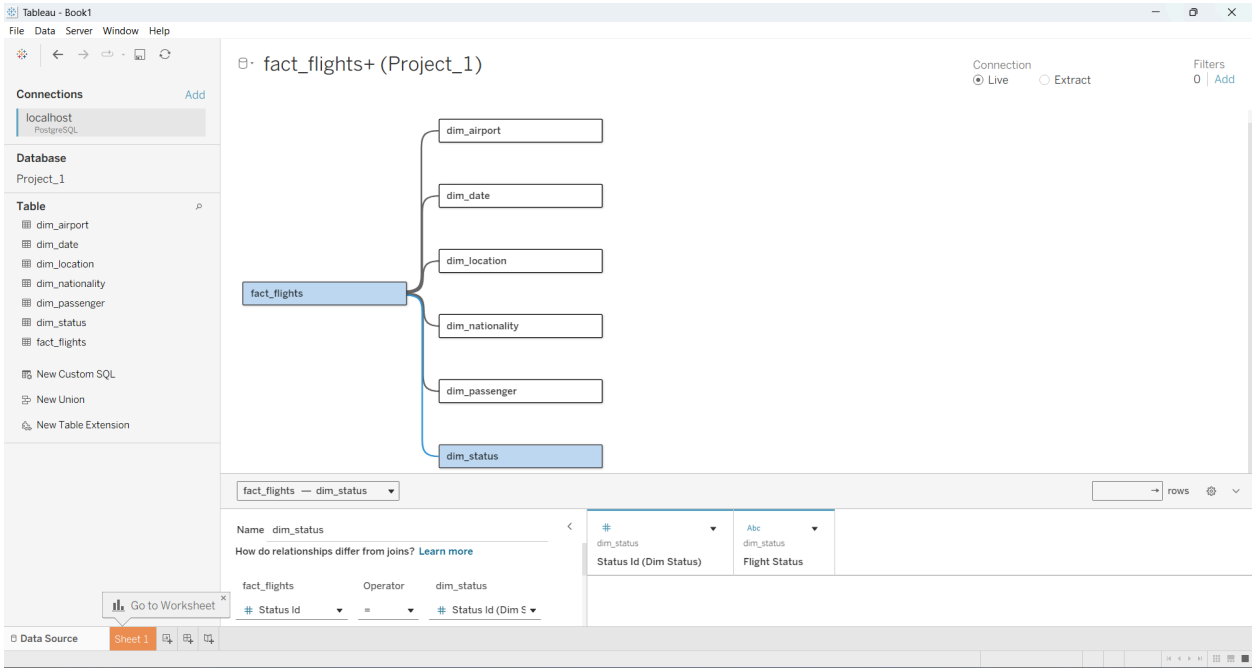


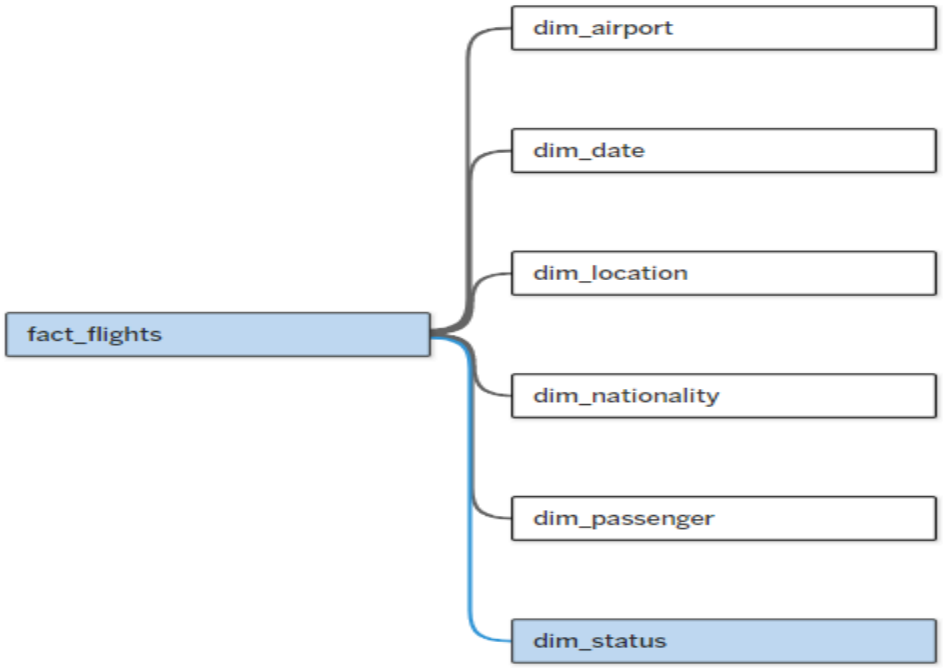


For PostgreSQL, we manually connect it by typing the server and database name.



Finally, we drag the tables into the data source and connect the fact tables with each dimension based on the unique identifiers for each dimension, primarily their corresponding primary-foreign key pairs. From this we can apply OLAP on Tableau from the data warehouse.


Tableau - Book1
File Data Server Window Help
Connections Add
localhost
PostgreSQL
Database
Project_1
Table
dim_airport
dim_date
dim_location
dim_nationality
dim_passenger
dim_status
fact_flights
New Custom SQL
New Union
New Table Extension
fact_flights+ (Project_1)
Connection
Live
Extract
Filters
0 Add
dim_airport
dim_date
dim_location
fact_flights
dim_nationality
dim_passenger
dim_status
fact_flights — dim_status
rows
Name dim_status
How do relationships differ from joins? Learn more
fact_flights
Operator
dim_status
Status Id
Status Id (Dim S
dim_status
Status Id (Dim Status)
dim_status
Flight Status
Go to Worksheet
Data Source
Sheet 1


dim_airport
dim_date
dim_location
fact_flights
dim_nationality
dim_passenger
dim_status

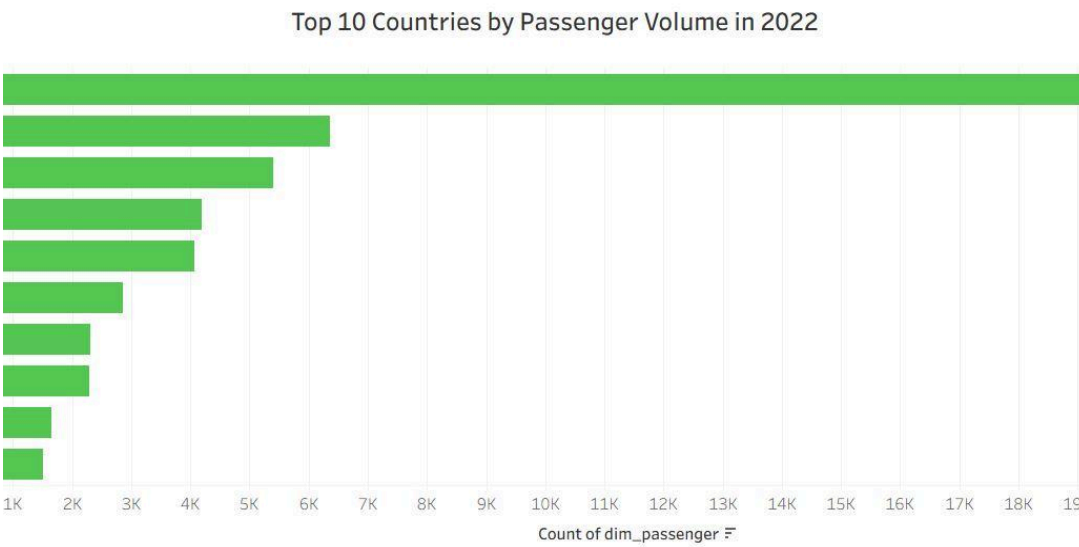
5. Visualizations of Queries

1. Which countries had the highest air travel demand in 2022. (Top-10)

```
SELECT
l.country_name,
COUNT(*) AS total_passengers
FROM Fact_Flights f
JOIN Dim_Location l
ON f.location_id = l.location_id
JOIN Dim_Date d
ON d.Date_ID = f.Date_ID
WHERE d.YEAR = 2022
GROUP BY l.country_name
ORDER BY total_passengers DESC
LIMIT 10;
```

Data Output | Messages | Notifications

	country_name character varying (200)	total_passengers bigint
1	United States	21728
2	Australia	6353
3	Canada	5393
4	Brazil	4192
5	Papua New Guinea	4068
6	China	2865
7	Indonesia	2311
8	Russia	2289
9	Colombia	1648
10	India	1519



Insights:

- When we analyze this we can identify the top 10 countries with the highest air travel demand in 2022.
- Using that we can know which countries recovered well after the Covid-19 pandemic and the Australian government can follow the new travel policies these countries came up with and use that knowledge to improve Australian policies.
- When we look at our data, the USA had the best recovery and had over 22000 passengers and Australia has performed well and became the second with nearly 6300 passengers.
- Even though that's good progress, there's a big gap between the first two countries so the Australian government can look at the new USA policies. Also, there are several other countries that perform well just as Australia, so those governments could have new innovative ideas that the Australian government can look at and use those policies in Australia.
- Australian airlines can also use these markets to look up about their seating capacity, avoiding over and under supply on routes and most importantly how those companies implemented the new post Covid-19 rules such as social distancing.

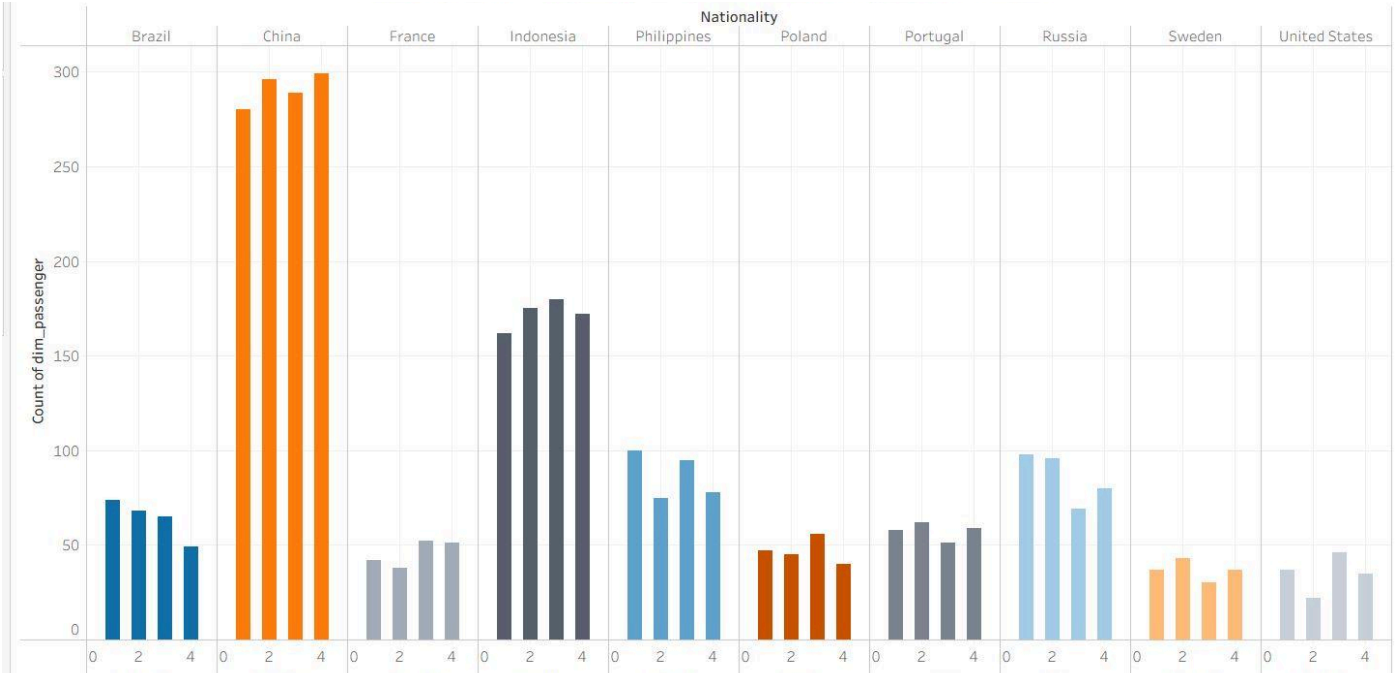
2. Which nationalities contributed the most to Australia's passenger foot traffic, and in which quarter do they travel most frequently ? (Top 10)

```sql
SELECT
n.Nationality,
COUNT(*) AS total_passengers
FROM Fact_Flights f
JOIN Dim_nationality n
ON f.Nationality_ID = n.Nationality_ID
JOIN Dim_Location l
ON f.Location_ID = l.Location_ID
WHERE l.Country_Name = 'Australia'
GROUP BY n.Nationality
ORDER BY total_passengers DESC
LIMIT 10;
```

Data Output | Messages | Notifications

	nationality character varying (50)	total_passengers bigint
1	China	1164
2	Indonesia	689
3	Philippines	348
4	Russia	343
5	Brazil	256
6	Portugal	230
7	Poland	188
8	France	183
9	Sweden	147
10	United States	140





Insights:

- This analysis reveals that the Chinese nationals are the largest contributor to Australia's inbound passenger foot traffic in 2022 highlighting the importance of the connection between China and Australia.
- Data also shows that there is a relationship between quarter and nationality. For example Chinese people tend to travel more in the 4th quarter which means at the end of the year compared to 1 st quarter which means the start of the year.
- The Australian tourism board can use these data to use their promotions and also come up with new promotions to improve their least performing quarter and not only that they can look up nationalities that weren't represented and create new tourism promotions for those nationalities.

3. Are flight delays and cancellation in Australia correlated with specific quarters ?

```
SELECT
d.Quarter,s.Flight_Status,
COUNT(*) AS total_passengers
FROM Fact_Flights f
JOIN Dim_Date d
ON f.Date_ID = d.Date_ID
JOIN Dim_Status s
ON f.Status_ID  = s.Status_ID
JOIN Dim_Location l
ON f.location_id = l.location_id
WHERE l.Country_Name = 'Australia'
GROUP BY d.Quarter,s.Flight_Status
ORDER BY d.Quarter;
```

Data Output | Messages | Notifications

	quarter integer	flight_status character varying (50)	total_passengers bigint
1	1	Cancelled	545
2	1	Delayed	526
3	1	On Time	526
4	2	Cancelled	526
5	2	Delayed	545
6	2	On Time	509
7	3	Cancelled	532
8	3	Delayed	538
9	3	On Time	543
10	4	Cancelled	523



Insights:

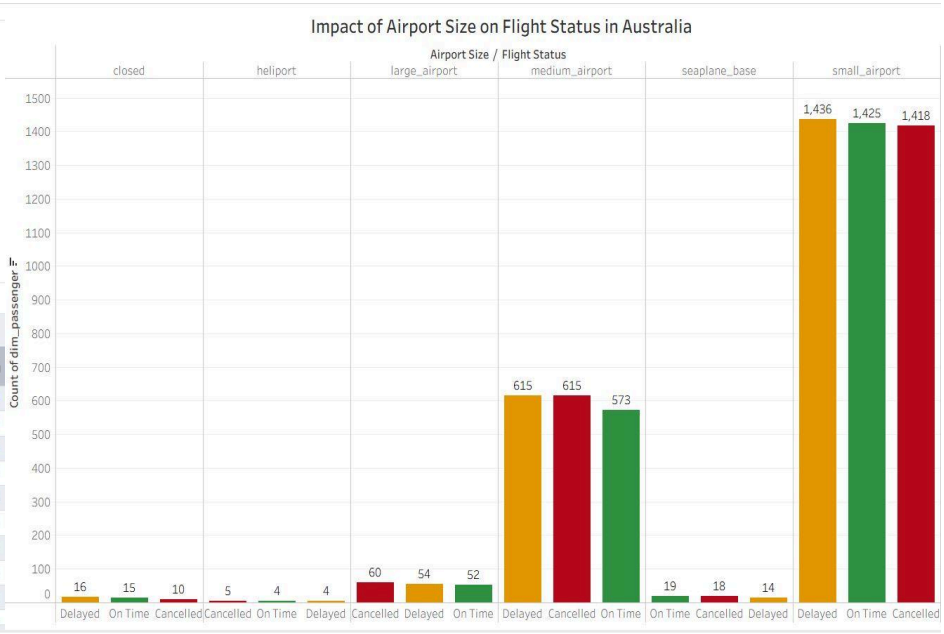
- This heatmap reveals clear quarterly patterns and different flight statuses.
- Quarter 3 of 2022 is a great example for that showing that it had multiple cancellations and delays while having low on time flights.
- The causes for these delays and cancellations could be due to unforeseen circumstances but also could have been due to some staff issues or passenger volume.
- Australian airports and governments can look into this and can allocate resources strategically whether it's more ground staff for a specific quarter, or maybe more personal check-in counters and other resources.

4. How does airport size (small, medium,large) across Australia influence the flight status ?

```
SELECT
a.Airport_Size,s.Flight_Status,
COUNT(*) AS total_passengers
FROM Fact_Flights f
JOIN Dim_Airport a
ON f.Airport_ID = a.Airport_ID
JOIN Dim_Status s
ON f.Status_ID  = s.Status_ID
JOIN Dim_Location l
ON f.location_id = l.location_id
WHERE l.Country_Name = 'Australia'
GROUP BY a.Airport_Size,s.Flight_Status
ORDER BY a.Airport_Size;
```

Data Output | Messages | Notifications

airport_size character varying (200)	flight_status character varying (50)	total_passengers bigint
closed	Cancelled	10
closed	Delayed	16
closed	On Time	15
heliport	Cancelled	5
heliport	Delayed	4
heliport	On Time	4
large_airport	Cancelled	60
large_airport	Delayed	54
large_airport	On Time	52
medium_airport	Cancelled	615



Insights:

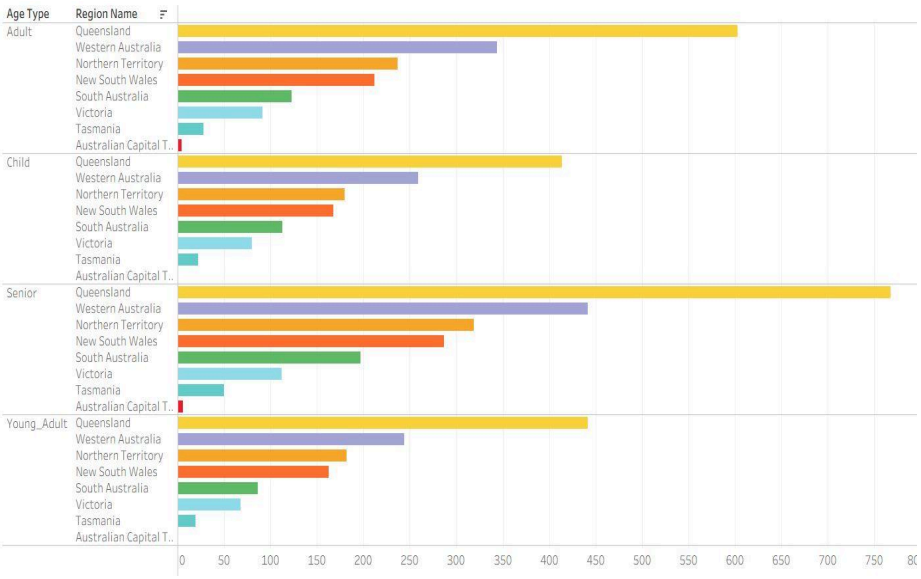
- This query and visualization demonstrate how there is a relationship between the airport size and flight statuses.
- When we look at the main three airport sizes of small, medium and large we can see there were a lot of passengers in those airports.
- Surprisingly larger and medium sized airports had higher cancellations and delays over on time flights while small airports had all three statuses in a much more equal manner.
- This can be simply due to operational complexity.
- The Australian government can use these findings to prioritize infrastructure investments such as adding additional gates and runaways extensions, automated check in systems and more and also, learn how different policies affect different airport sizes and making sure any airport can provide a reliable service no matter its size.

5. Which age group generates the highest foot traffic across Australian states/ regions ?

```
--Q5
SELECT
p.Age_Type,l.Region_Name,
COUNT(*) AS total_passengers
FROM Fact_Flights f
JOIN Dim_Passenger p
ON f.Passenger_Key = p.Passenger_Key
JOIN Dim_Location l
ON f.location_id = l.location_id
WHERE l.Country_Name = 'Australia'
GROUP BY p.Age_Type,l.Region_Name
ORDER BY total_passengers DESC;
```

Data Output | Messages | Notifications

	age_type character varying (50)	region_name character varying (200)	total_passengers bigint
1	Senior	Queensland	771
2	Adult	Queensland	607
3	Senior	Western Australia	447
4	Young_Adult	Queensland	443
5	Child	Queensland	417
6	Adult	Western Australia	353
7	Senior	Northern Territory	321
8	Senior	New South Wales	291
9	Child	Western Australia	265
10	Young_Adult	Western Australia	249



Insights:

- This visualization of horizontal group bar charts highlights which age dominates each Australian region/ state.
- Almost all the different age groups seem to prefer travelling to Queensland while seniors prefer the most.
- Western Australia is a close second and adults and young adults both tend to enjoy time in WA and it has a little difference with Queensland.
- The Australian government can use this information to make sure that each region has its own infrastructure to support these groups.
- Like for example since Queensland has been the major hub they have to make sure there's services for the seniors and adults while there are fun safe things for children and young adults.
- Something they can also focus on is that since WA is becoming popular with youths and adults they can have more festivals and sport competitions and other stuff that someone from that age group can enjoy so Queensland can become the hub for seniors and WA can be the place for the youth.

6. Association Rules Mining

The complete association mining process is implemented in the Project_1_association_mining.ipynb

Overview:

For this section, we are applying association rule mining to our airline dataset to uncover co-occurance patterns within passenger flight events. Moreover, the dataset combines passenger demographics, airport characteristics, geographic locations and flight status information, this provides a rich foundation for discovering relationships not easily visible through basic descriptive statistics.

The main goal is to find frequent attribute combinations within the airline dataset by applying Apriori algorithm. The dataset will first be transformed into transactional format, where each transaction represents a passenger's flight-event containing categorical descriptors. After, frequent itemsets are generated based on a minimum threshold to ensure that only statistically significant patterns are retained. Lastly, from these frequent itemsets, association rules are implemented and evaluated using support, confidence and lift. These metrics will then help distinguish between rules that occur simply to dataset size or actual genuine behavior or operational patterns. These insights can help support business decision-making by revealing patterns that may occur more frequently than others, highlighting potential relationships between passenger demographics, airport characteristics and geographic location.

Libraries and Set-up

- Pandas and Numpy: data handling and mathematical operations
- mlxtend: Library used that provides the ff.:
    - TransactionEncoder: converts transactions into binary matrix format
    - Apriori: used to apply apriori algorithm to find frequent itemsets
    - Association_rules: generate the association rules from the frequent itemsets and calculate support, confidence and lift.

```
# Importing libraries
import pandas as pd
import numpy as np
import mlxtend
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules
```

Methodology:

Data Preprocessing

We first read the dataset we cleaned and exported from the ETL section.

```
df = pd.read_csv('Final_Dataset.csv')
```

```
df
```

	Passenger_ID	Gender	First_Name	Last_Name	Age	Nationality	Airport_Name	Departure_Date	Flight_Status	Airport_Size	Continent	Country_N
0	ABVWIg	Female	Edithe	Leggis	62	Japan	Coldfoot Airport	2022-06-28	On Time	small_airport	North America	United S
1	jkXXAX	Male	Elwood	Catt	62	Nicaragua	Kugluktuk Airport	2022-12-26	On Time	small_airport	North America	Ca
2	CdUz2g	Male	Darby	Felgate	67	Russia	Grenoble Alpes Isère Airport	2022-01-18	On Time	medium_airport	Europe	Fr
3	BRS38V	Female	Dominica	Pyle	71	China	Ottawa / Gatineau Airport	2022-09-16	Delayed	medium_airport	North America	Ca
4	9kvTLo	Male	Bay	Pencost	21	China	Gillespie Field	2022-02-25	On Time	small_airport	North America	United S

This step is important, as we need to load in the dataset and remove any unnecessary identifier columns. We remove the Passenger_ID column

```
# dropping identifier columns
df = df.drop('Passenger_ID', axis = 'columns')
```

```
df
```

	Gender	First_Name	Last_Name	Age	Nationality	Airport_Name	Departure_Date	Flight_Status	Airport_Size	Continent	Country_Name	Region_N
0	Female	Edithe	Leggis	62	Japan	Coldfoot Airport	2022-06-28	On Time	small_airport	North America	United States	A
1	Male	Elwood	Catt	62	Nicaragua	Kugluktuk Airport	2022-12-26	On Time	small_airport	North America	Canada	Nu
2	Male	Darby	Felgate	67	Russia	Grenoble Alpes Isère Airport	2022-01-18	On Time	medium_airport	Europe	France	Auve Rhône-
3	Female	Dominica	Pyle	71	China	Ottawa / Gatineau Airport	2022-09-16	Delayed	medium_airport	North America	Canada	Qu
4	Male	Bay	Pencost	21	China	Gillespie Field	2022-02-25	On Time	small_airport	North America	United States	Cali

Binning: Next, we decide to bin certain categorical features by grouping into meaningful ranges, we will group airport sizes into Commercial and Non-Commercial airports. We do this by specifying small, medium and large airports as Commercial airports and everything else as Non-Commercial Airports. These transformations reduce category fragmentation and should increase support values and generate more meaningful association rules.

```
def airport_type(airport):
    if airport in ['small_airport', 'medium_airport', 'large_airport']:
        return 'Commercial'
    else:
        return 'Non-Commercial'

df['Airport_Type'] = df['Airport_Size'].apply(airport_type)
```

```
df
```

st_Name	Last_Name	Age	Nationality	Airport_Name	Departure_Date	Flight_Status	Airport_Size	Continent	Country_Name	Region_Name	Airport_Type
Edithe	Leggis	62	Japan	Coldfoot Airport	2022-06-28	On Time	small_airport	North America	United States	Alaska	Commercial
Elwood	Catt	62	Nicaragua	Kugluktuk Airport	2022-12-26	On Time	small_airport	North America	Canada	Nunavut	Commercial
Darby	Felgate	67	Russia	Grenoble Alpes Isère Airport	2022-01-18	On Time	medium_airport	Europe	France	Auvergne-Rhône-Alpes	Commercial
Dominica	Pyle	71	China	Ottawa / Gatineau Airport	2022-09-16	Delayed	medium_airport	North America	Canada	Quebec	Commercial
Bay	Pencost	21	China	Gillespie Field	2022-02-25	On Time	small_airport	North America	United States	California	Commercial

Feature Selection:

We will now select meaningful attributes from the preprocessed dataset to apply the association rule mining. The reason is that we want to identify interpretable relationships within the dataset.

Criteria:

- Categorical: The features selected are categorical
- Interpretability: Features must be easily understandable and interpretable
- Relevance: Features selected must capture key aspects for flights

```
# add the header and then the class, this is for easier reading once we get the results
columns = ['Age_Type', 'Gender', 'Airport_Type', 'Continent' ,'Flight_Status']
new_df = df[['Age_Type', 'Gender', 'Airport_Type', 'Continent' ,'Flight_Status']]
for col in columns:
    new_df[col] = col +'=' + new_df[col].astype(str)
```

```
new_df.head(4)
```

	Age_Type	Gender	Airport_Type	Continent	Flight_Status
0	Age_Type=Senior	Gender=Female	Airport_Type=Commercial	Continent=North America	Flight_Status=On Time
1	Age_Type=Senior	Gender=Male	Airport_Type=Commercial	Continent=North America	Flight_Status=On Time
2	Age_Type=Senior	Gender=Male	Airport_Type=Commercial	Continent=Europe	Flight_Status=On Time
3	Age_Type=Senior	Gender=Female	Airport_Type=Commercial	Continent=North America	Flight_Status=Delayed

Apriori Algorithm:

We use the Apriori algorithm, introduced by Agrawal and Srikant(1994) to perform association rule mining. This works by identifying the all single items and filters by minimum support threshold and extending them into a larger itemset, pruning any itemset whose subsets are infrequent. Once frequent itemsets are generated, association is derived based on a given minimum confidence threshold. To implement this, we import the library mlxtend into Python, which contains the function Apriori and association_rule. These functions make generating

itemsets more efficient and easier, as well as easily compute the support, confidence and lift. These 3 metrics are explained as follows:

- Support: This measures how frequent an itemset appears in the dataset. A higher support indicates the itemset appears more often in the dataset, while a lower one indicates it is a rare event.
- Confidence: This is the conditional probability when the consequent is true, given the antecedent is true. A high confidence indicates that the consequent happens more often when the antecedent occurs. However this does not indicate any meaningful relation/pattern, since it can be inflated given the consequent appears more frequently in the dataset.
- Lift: This measures how much more likely the consequent will happen when the antecedent occurs, compared to how often the consequent occurs overall. A lift value greater than 1 indicates a positive correlation, a lift value less than 1 indicate a negative correlation and a lift value equal to 1 indicates the consequent and antecedent are independent.

Association Rule Mining:

The first step prior to applying association rule mining is to transform the data type into string. The reason is that the TransactionEncoder function only handles string type variables. We then create a list from the datasets and apply one-hot-encoding on the list, because Apriori function only allows boolean data types only. Lastly, we reconvert them back into a dataframe.

```
#Data transformation
#TransactionEncoder() function only can handle string type
new_df = new_df.astype(str)

#TransactionEncoder() was designed to covert lists to array
list = new_df.values.tolist()

#Covert the list to one-hot encoded boolean numpy array.
#Apriori function allows boolean data type only, such as 1 and 0, or FALSE and TRUE.
te = TransactionEncoder()
array_te = te.fit(list).transform(list)

#Check the array
array_te

#Check the colunms
te.columns_

#Apriori function can handle dataframe only, covert the array to a dataframe
arm_df = pd.DataFrame(array_te, columns = te.columns_)
```

Next, we implement the Apriori algorithm to find the frequent itemsets from the encoded dataset first. We use a minimum threshold of 0.1 for a more detailed and granular pattern from the dataset. A higher threshold would have restricted the analysis to the most common dataset and

would have overlooked less frequent but relevant combinations of attributes. This should give us a more detailed comprehensive look into the dataset.

```
#Find the frequent itemsets
frequent_itemsets = apriori(arm_df,min_support=0.1,use_colnames =True)

#Check the length of rules
frequent_itemsets['length']=frequent_itemsets['itemsets'].apply(lambda x: len(x))

rules = association_rules(frequent_itemsets, metric='lift', min_threshold=1.0)
flight_status_rules = rules[rules['consequents'].apply(lambda x: all('Flight_Status=' in item for item in x))]
top_k_rules = flight_status_rules.sort_values(by=['lift','confidence'], ascending=False).head(3)
```

```
top_k_rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
```

Top K Rules:

In association rule mining, k-values refer to the top rules selected based on the specific metrics. This refers to the k most interesting association based on certain evaluation metrics. Where, k is the number of top rules selected for analysis. For this project, the 'rule' takes the form of antecedent(LHS) → consequent(RHS), that is the left hand side represents the condition and the right hand side represents the outcome being predicted. This project deliberately placed Flight Status on the RHS, meaning we want the rule to determine the flight status outcome given the LHS. Therefore, Top K rules represent the most interesting rules that predict flight status, and we arrange them in descending order by lift then confidence. The k-value is set at 3, meaning we select the top 3 most interesting associations to analyse.

```
pd.set_option('display.max_colwidth', None)
print(top_k_rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

```
                                        antecedents  \
57  frozenset({Airport_Type=Commercial, Age_Type=Senior})
7                          frozenset({Age_Type=Senior})
85     frozenset({Gender=Male, Airport_Type=Commercial})

                         consequents   support  confidence      lift
57  frozenset({Flight_Status=Delayed})  0.109033    0.335353  1.007254
7   frozenset({Flight_Status=Delayed})  0.115057    0.335116  1.006541
85  frozenset({Flight_Status=On Time})  0.159543    0.334747  1.005030
```

Results and Interpretation:

Rule 1: Airport_Type=Commercial AND Age_Type=Senior ⇒ Flight_Status=Delayed

This rule suggests that senior passengers who are travelling through commercial airports have a tendency to experience delayed flights. Support of 0.109033 means that about 10.90% of flights from the dataset match this pattern. Confidence suggests that when a senior passenger is travelling through a commercial airport there is a 33.53% chance that the flight will be delayed. Lastly, lift of 1.007254 shows that senior passengers using commercial

airports are 0.7254% more likely to experience delays compared to the overall dataset average, this is very negligible.

Rule 2: Age_Type=Senior ⇒ Flight_Status=Delayed

This rule suggests that senior passengers have a tendency to experience delayed flights. Support of 0.115057 means that about 11.51% of flights from the dataset match this pattern. Confidence suggests that when a passenger is a senior there is a 33.51% chance that the flight will be delayed. Lastly, lift of 1.006541 shows that seniors are 0.6541% more likely to experience delays compared to the overall dataset average, this is very negligible.

Rule 3: Gender=Male AND Airport_Type=Commercial ⇒ Flight_Status=On Time

This rule suggests that male passengers travelling in a commercial airport have a higher chance to experience on time flights. Support of 0.159543 means that about 15.95% of flights from the dataset match this pattern. Confidence suggests that when a passenger is a male and travelling in a commercial airport there is a 33.47% chance that the flight will be on time. Lastly, lift of 1.005030 shows that male passengers who are travelling in a commercial airport are 0.5030% more likely to experience on time flights compared to the overall dataset

Insights and Discussion:

While we were able to extract the top 3 rules with flight status on the RHS, the mining actually does not reveal any meaningful or strong relationships/patterns. This can be seen from several results we found.

- Negligible Lift Values: all lift values are between 1.005 to 1.007, which are close to 1, suggesting the LHS and RHS are near independent from each other.
- Low Confidence Values: all confidence values are around 33.5%, meaning the rule only correctly predicts flight status 1 in 3 times. This is barely good as a random guess since the flight status has 3 class outcomes.
- Listed attributes may be insufficient as predictors of flight status, and perhaps flight status is better captured by other attributes not in the dataset.
- Lack of granularity of dataset: the other attributes are grouped by broader categories, which grouped a large number of passengers together, making it difficult for the algorithm to detect more meaningful patterns.
- Class imbalances in flight status: It's also possible that a specific class in flight status appears more often than the rest, which forces the algorithm to favour rules towards that specific class.

While the Apriori algorithm and association mining rule was correctly implemented, the overall dataset attribute may not be well-suited in predicting flight status though association mining rules, and may require the use of other attributes that may contribute more in yielding meaningful associations.

Recommendations:

- **Improve flight punctuality and delay management**: Given that association mining has uncovered that seniors are experiencing delays more often, there should be established policies to monitor and track flights and publish these to the public. Moreover, airlines with high delay rates should be penalized and must improve operational structure to drive improvements in on-time performance.
- **Improve commercial airport infrastructure**: Most of the associations are involved in commercial airports, these airports should be modernized and renovated to improve efficiency. Moreover, fast-track boarding lanes, increased gate capacity, and automated check-in and luggage system should be deployed in order to minimize delays from operational bottlenecks.
- **Mandate aviation data transparency**: Given how the association mining does not reveal any meaningful/strong relationships, governments should implement a data transparency act against airlines. These records should be placed in a centralized database so that deeper analysis and more targeted policies can be developed to improve the aviation operations.

7. Conclusion

This project analyzed the airline data from the year 2022 to evaluate how air travel performed and recovered after the Covid - 19 pandemic and to provide useful insights to Australian governments to make more data driven decisions on improving their tourism and aviation sectors. We created a data warehouse, used OLAP and association mining techniques to analyze flight operations and passenger behaviors.

We were able to find some key insights such as, USA and Australia showing the strongest recovery in air travel post pandemic, Chinese national being the highest inbound passenger traffic to Australia showing how much international market can have an impact, different quarters of the year having different patterns in flight statuses, larger and medium airports having higher disruption rates compared to smaller ones and more. One of the most surprising findings was that no matter what age group they were from, Queensland, the region in Australia, was the most popular destination among all of them. So, based on the findings we can see that international markets, age, quarters and even airport size can have an impact on air travel.

Even though the OLAP approach was able to find meaningful insights and patterns, we faced several limitations when we were working on this project. Limitations such as, dataset lacking granularity and not having more specific key variables to come up with measurements and not being able to detect more precise patterns. That limitation also affected our association mining because it didn't have any strong actionable relationship and the results had low lift and confidence values.

So based on our analysis and findings we can recommend that, Australian government should put more resources and develop the infrastructures of their medium and large airports to reduce the delays and cancellations. That can be through hiring more ground staff, self checking points and more. Other than that, what they can do is to have more targeted tourism strategies through developing better relationships with countries such as China and Indonesia who have a big

impact on their tourism. But not only that, what they can do is focus on the countries that's not on the top and try to understand what they can do to attract those countries. Another thing that they can focus on is that how everyone preferring Queensland and so making sure they have a good experience on Queensland but also what they can do is check the second preferred region like youth preferring western Australia, so Australian government can focus on making WA a hub for youth through festivals and other events that can attract youth.

Furthermore, to have a deeper analysis on these topics they can make the aviation data more transparent and by using those data we can apply more advanced analytics such as predictive modeling to understand air travel data even more.

References

1. J. Han, J. Pei, and M. Kamber, Data Mining: Concepts and Techniques, 3rd ed. Elsevier/Morgan Kaufmann, 2012.
2. R. Kimball and M. Ross, The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd ed. Wiley, 2013.
3. R. Kimball, The Data Warehouse ETL Toolkit. Wiley, n.d.

Appendix

Use of Gen AI in report preparation

- Age_Type generated in the fully joined dataset or during dimension creation
    - AI Suggestion: AI suggested we include these new features during dimensional modeling, the reason for this is data consistency and avoid redundancy in later pipelines.
    - Decision: We followed AI's suggestion in order for data consistency and reduce any redundancy.
- Separating Nationality and Flight Status to its own respective dimensions
    - AI Suggestion: AI suggested to place Nationality and Flight status attributes separately, as they are descriptive and should be stored separately for cleaner and easier filtering for OLAP.
    - Decision: Agreed, we want our dimension to be inline with Kimball's dimension modeling principles. Separating them into their own dimension should make drill-down, roll-up and slice and dice operations more accessible.
- Feature engineering for Association Mining
    - AI Suggestion: AI suggested attributes with smaller classes should be used, since too many classes could dilute the support values and weaken the rules

discovered. Also, they suggested grouping airport size to airport type, since certain airport sizes are used for non-commercial flights.
    - Decision: Agreed. Along with the requirements of the RHS needed to be flight status, we selected the attributes and engineered features that should be meaningful in finding patterns towards flight status.
- Association Rule Mining Understanding of Concept
    - AI Suggestion: AI was consulted to further deepen our understanding on Apriori algorithm, association mining, the metrics support, confidence and lift, and k-values.
    - Discussion Outcome: With the help of AI, we were able to understand deeper how apriori generate itemsets, how we should interpret the metrics, the terminology of RHS and LHS. From this, we were able to produce an outcome that meets project specification and able to discuss our opinions on the results.
- Policy recommendation structure
    - AI Suggestion: AI suggested we structure our recommendation inline with the results found in association mining, even when the association mining results were negligible, ensuring the recommendations are still grounded and justifies the rules found from association mining.
    - Decision: Adopted. This gives us concrete and targeted-based recommendations from the discovered rules.